Closest Pairs Search Over Data Stream

ABSTRACT

k-closest pair (KCP for short) search is a fundamental problem in the field of database. Let $\mathcal{S}$ be a set of d-dimensional streaming data. Every two objects o_i and o_j in $\mathcal{S}$ construct a pair. The goal of KCP search is to retrieve k pairs with the shortest distances. Some existing works study continuous 1-closest pair query (i.e., $k = 1$) over dynamic data environment where object insertions/deletions are allowed. Unfortunately, all these algorithms incur very high computational cost, and they cannot be easily extended to support KCP search with $k > 1$. In this paper, we study the problem of KCP search over turnstile stream model. Our target is incrementally maintaining, as few as, pairs formed by streaming data to support KCP search with arbitrarily parameter k. In order to achieve it, we first introduce the concept of NNS (short for Nearest Neighbour pair-Set), which consists of all the nearest neighbour pairs. It allows us to support KCP search via only accessing $O(k)$ objects. We further observe that in most cases we only need to use a small set of NNS to answer KCP search as typically $k \ll n$. This observation motivates us to explore the idea of maintaining a subset of "high quality" NN pairs, i.e., pairs with the shortest distances, to answer KCP search. Accordingly, we propose TNNS (short for Threshold-based NN pair Set) that contains a small number of NN pairs and a partition named τ-DLBP (short for τ-Distance Lower-Bound based Partition) to organize objects with τ being an integer much smaller than n. τ-DLBP organizes objects using up to $O(\log \frac{n}{\tau})$ partitions and is able to support the construction and update of TNNS efficiently.

CCS CONCEPTS

• **Computer systems organization** → **Embedded systems**; *Redundancy*; Robotics; • **Networks** → Network reliability.

KEYWORDS

Streaming Data, $k-$Closest Pair Search, Partition, Cube

ACM Reference Format:

. 2018. Closest Pairs Search Over Data Stream. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX).* ACM, New York, NY, USA, 16 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In this paper, we study the problem of k-closest pairs search (KCP search in short) [1][2][3][4] over data stream. When $k = 1$, it looks for a pair of two objects with the smallest mutual distance, among the given n objects in d-dimensions. We study a more general version of the original closest pair search by allowing $k > 1$ and we consider a dynamic setup where objects are from a data stream environment.

Formally, let $\mathcal{S}$ be a d-dimensional streaming data set. Every two objects o_i and o_j construct a pair, denoted by $p(o_i, o_j)$. The goal of KCP search is to retrieve k pairs with smallest scores among all pairs constructed by objects in $\mathcal{S}$. Here, the score of a pair $p(o_i, o_j)$ is determined by a scoring function, dependent on the needs of different applications. We adopt a distance function $\mathsf{D}(o_i, o_j)$, that measures the Euclidean distance between objects o_i and o_j, in this paper for illustration. Without loss of generality, we study this problem under turnstile stream model [5][6][7], in which objects insertion and deletion are allowed. In addition, objects' arrival order may be very different from their expiry order.

KCP search is a fundamental yet challenging problem that has been studied over thirty years. It has to take into account a potentially large number of pairs. For example, given a dataset $\mathcal{S}$ of n objects, each object o constructs $O(n)$ pairs with other objects in $\mathcal{S}$, in total $O(n^2)$ pairs. The dynamic feature of data stream aggravates the difficulty of KCP search, where a single update affects all objects in $\mathcal{S}$, i.e., an insertion triggers the construction of $O(n)$ pairs and a deletion triggers the expiry of $O(n)$ pairs.

KCP search has many applications over turnstile data stream. For example, an online carpooling booking system could find suitable users for car-sharing based on their submitted *pick-up point*s, *drop-off point*s and *planned pick-up time* via KCP search. Users' requests could be regarded as turnstile data stream. In other words, requests flow into the system at the moment they are submitted and expire when they are served or their corresponding planned pick-up time is reached, i.e., the arrival order of requests may be very different from the expiry order. KCP search also could be applied in logistics management system (packages grouping) and stock recommendation (search for similar stocks), to name but a few.

To the best of our knowledge, none of existing approaches could support KCP search over data stream. The closest work is the study on continuous 1CP ($k = 1$) query over dynamic data environment, in which objects insertion and deletion are allowed. Based on the ways objects are organized, the main efforts on continuous 1CP query over dynamic data can be categorized into two groups, namely *tree-based approaches* and *partition-based approaches*. The former [8][9] uses a tree-based index to maintain objects, and meanwhile monitors nearest neighbour (NN) of every object. When objects are inserted into or deleted from the dataset, both the index and the NN of related objects are updated. Unfortunately, the cost of maintaining every object's NN is very high. The latter [10] uses random algorithms to partition objects into a group of subsets based on distances (or distance lower-bound) between objects and their NNs. Compared with tree-based approaches, they avoid maintaining every object's NN. However, the cost of processing an insertion or a deletion is $O(n)$ in the worst case.

Above all, the computational cost of both types of algorithms is very high and hence they cannot be easily extended to perform the search when $k > 1$ under data stream. On one hand, algorithms

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

have to find query result pairs from a larger scale of pairs other than just the NN pairs constructed by objects and their NNs. On the other hand, objects' insertion/deletion frequency under data stream is much higher than that under dynamic data environment, and it is difficult for them to process multitudes of newly arrived/expired objects in real time. Consequently, it is essential to propose an approach that is able to process KCP search quickly and meanwhile support efficient update under data stream.

Solution Overview. Our solution is motivated by the following findings. Given an object $o \in \mathcal{S}$, if its NN pair is not part of the answer set to a KCP, it is guaranteed that all the pairs containing o will NOT be part of the answer set either. We further prove that, for a given KCP, we can find the result pairs via only considering distances between objects that form k NN pairs with the shortest distances. Accordingly, we form the set NNS (short for Nearest Neighbour pair Set), which consists of all NN pairs. The KCP search algorithm developed based on NNS is able to support KCP search with the cost of $O(k^2)$. We then propose an index named QC-Tree to ease the construction and update of NNS. QC-Tree, though similar to quad-tree, is designed to have its height bounded by $O(\log n)$, regardless of the distribution of objects. In addition, when searching for objects' NNs, the leaf nodes reflects the upper-bounds of distances between objects and their NNs, which allow us to convert the expensive NN search to cheaper range queries.

However, the maintenance of NNS is expensive because the computational cost of incrementally maintaining every object's NN is high under data stream, i.e., $O(n)$ per insertion/deletion in the worst cases. We observe that in most cases we only need to use a small set of NNS to answer KCP search as typically $k \ll n$. This observation motivates us to explore the idea of maintaining a subset of "high quality" NN pairs, i.e., pairs with the shortest distances, to answer KCP search. Accordingly, we propose TNNS (short for Threshold-based NN pair Set) that contains a small number of NN pairs, guided by a controlling parameter τ.

To incrementally maintain TNNS under data stream, we propose a novel partition named τ-DLBP (short for τ-Distance Lower-Bound based Partition) to organize objects using up to $O(\log \frac{n}{\tau})$ partitions. It uses one partition to maintain objects that contribute to TNNS, and uses other partitions to organize the rest of objects based on the lower bounds of their distances to their NNs. As compared with QC-Tree, τ-DLBP significantly reduces the update cost to $O((d + 3^d) \log \frac{n}{\tau} + \tau)$ per update. In addition, τ-DLBP organizes objects based on their likelihood of contributing to KCP search and its partitions have very different sizes, i.e., objects having shorter distances to their NNs are inserted into smaller partitions with tighter distance lower bounds. In this way, when a KCP search is submitted, if k is small, we only need to access the partition containing TNNS to perform the search directly; even if k is large, we can retrieve query result pairs via accessing objects located at a few partitions only.

The rest of the paper is organized as follows. Section 2 formally presents the concept of NNS, together with its construction, updating and KCP search algorithm. Section 3 proposes TNNS and presents τ-DLBP partition framework. Section 4 reports the results of our experimental evaluation. Section 5 reviews the previous work. Finally, Section 6 concludes the paper with a summary of our findings.

Table 1: Frequent Notations

$\mathcal{S}$	a streaming dataset with n objects in $[0,1]^d$ space
$NN(o)$	the nearest neighbor of an object o
$RNN(o)$	the reverse nearest neighbor(s) of an object o
$\mathsf{D}(o)$	the distance between o and its nearest neighbor, also termed as the *score* of object o
$\mathsf{D}(o, o')$	the distance between two objects o and o'
$q(o, r, S')$	a range search to locate objects in S' that are within r distance to o

2 SUPPORTING KCP VIA NN PAIR SET

As stated in Section 1, NN pairs play an important role in KCP search. In this section, we first formally introduce the concept of NNS (short for Nearest Neighbour pair Set); then explain how NNS could facilitate KCP search; and finally present an index structure that is able to construct NNS for a given dataset and to maintain NNS in a dynamic environment. Table 1 defines the list of frequently used notations throughout this paper.

2.1 Supporting KCP Search Via NNS

DEFINITION 1. **NNS**. *Given a streaming data set $\mathcal{S}$, NNS $\mathcal{N}$ ($\subset \mathcal{S} \times \mathcal{S}$) consists of all the NN pairs, i.e., $\mathcal{N} = \{p(o, NN(o)) | o \in \mathcal{S}\}$.*

LEMMA 1. *Given a streaming dataset $\mathcal{S}$, let $\mathcal{N}$ be the NNS and R_k refer to the result set of a KCP query. If $k = 1$ (i.e., closest pair search), it is guaranteed that $R_1 \subset \mathcal{N}$ (See [8]).*

As stated in Lemma 1, the result to a 1CP query must be contained in NNS. In addition, once we have the result set R_i to the KCP search with $k = i$, we can support KCP search with $k = (i + 1)$ by finding the next target result pair $p(o, o')$, assuming $R_{i+1} = R_i \cup \{p(o, o')\}$ via the above two cases, as listed below. Here, $R_i.obj$ refers to the set of objects that appear in at least one result pair in R_i.

Case 1: Both objects are in $R_i.obj$, i.e., $o \in R_i.obj \wedge o' \in R_i.obj$. Given R_i, we can find the next target pair via scanning all the pairs formed by the objects in $R_i.obj$, i.e., $\{p(o_a, o_b) | o_a \in R_i.obj \wedge o_b (\neq o_a) \in R_i.obj \wedge p(o_a, o_b) \notin R_i\}$.

Case 2: Otherwise, $p(o, o')$ belongs to $(\mathcal{N} - R_i)$, as stated in Lemma 2.

LEMMA 2. *Given a streaming dataset $\mathcal{S}$, let R_i and R_{i+1} ($= R_i \cup \{p(o, o')\}$) be the two result sets to KCP queries corresponding to $k = i$ and $k = (i + 1)$, respectively. It is guaranteed that if $o \notin R_i.obj \vee o' \notin R_i.obj$, $p(o, o') \in \mathcal{N} - R_i$.*

PROOF. See APPENDIX-A □

Let NN_k be the set of top-k NN pairs with the shortest distances, and $NN_k.obj$ ($|NN_k.obj| \leq 2k$) be the set of objects whose NN pair belongs to NN_k. The above analysis implies that we can support KCP search via only considering pairs formed by objects in $NN_k.obj$. Take Figure 1 as an example. We use a directed graph to represent the nearest neighbour relationships between objects, e.g., NN of o_3 is o_1, so an in-edge with weight $\mathsf{D}(o_3, o_1)$=0.05 from o_3 to o_1 is constructed. Accordingly, we have $NN_1 = \{p(o_1, o_2)\}$, $NN_2 = \{p(o_1, o_2), p(o_1, o_3)\}$, and so on. Given a KCP search with $k = 1$, we can immediately return the answer result $p(o_1, o_2)$ based on Lemma 1. Given another KCP search with $k = 2$, we can answer it via objects

Algorithm 1: The KCP Search Algorithm

Input: dataset $\mathcal{S}$, NN-Core $\mathcal{N}$, k
Output: Query Result Set R

1 $R \leftarrow \emptyset$, min-heap $H \leftarrow$ construct($\mathcal{N}, k$);
2 **while** $|R| < k$ **do**
3 Pair $p(o, o') \leftarrow$ pop(H);
4 **if** $o \notin R.obj$ **then**
5 **for** *each object* $o_j \in R.obj$ **do**
6 **if** $D(o_j, o) < D(H_{max})$ **then**
7 $H \leftarrow H - H_{max}, H \leftarrow H \cup p(o_j, o)$;
8 repeat Lines 4-7 to process o';
9 $R \leftarrow R \cup \{p(o, o')\}$;
10 return R;

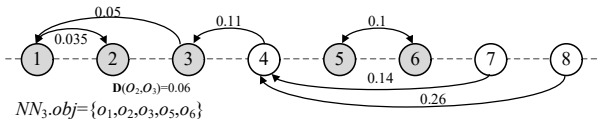


Figure 1: 3-closest pair search under NNS

in NN_2 (i.e., $\{o_1, o_2, o_3\}$). To be more specific, in total three pairs could be formed by objects in NN_2, and $\{p(o_1, o_2), p(o_1, o_3)\}$ forms the result set because of shorter distances. If $k = 3$, we consider pairs formed by objects in NN_3 (i.e., $\{o_1, o_2, o_3, o_5, o_6\}$) and return the three pairs with the shortest distances (i.e., $\{p(o_1, o_2), p(o_1, o_3), p(o_2, o_3)\}$) as the result set.

Following the two cases presented above, we propose a KCP Search Algorithm with its pseudo-code listed in Algorithm 1. Assuming NN pairs in $\mathcal{N}$ are sorted based on their scores, this search algorithm can complete the search within $O(k^2)$. Note, H is a min-heap that keeps up-to k candidate pairs with smallest scores among the object pairs that have been explored so far, and R is the result set to be returned by the algorithm. Initially, R is set to $\emptyset$ and H is constructed by the top-k object pairs in $\mathcal{N}$ with minimum scores (Line 1). The initialization of H with object pairs in $\mathcal{N}$ is to make sure all the result pairs belonging to Case 2 have been well considered. Thereafter, we incrementally update the result set R by locating the next target result pair and updating H, until R contains k result pairs. For easy presentation, function $pop(H)$ is to pop out the head entry with minimum score from the min-heap H, H_{max} refers to the object pair in H that has the maximum score, and $D(H_{max})$ refers to the score of H_{max}. In each iteration, the top entry $p(o, o')$ of H is inserted into R as the next target result pair (Lines 3, 9). E.g., when $R = \emptyset$, the pair in H with minimum score is inserted into R as the result pair to 1CP query, as stated in Lemma 1. Thereafter, we update H based on the candidate pairs newly introduced by the target result pair $p(o, o')$, i.e, pairs constructed by an object in $R.obj$ and object o or o'. The algorithm is terminated when $|R|$ reaches k.

2.2 The Construction of NNS

As stated above, the concept of NNS could effectively facilitate the processing of KCP search. In the following, we will explain how to construct NNS. For simplicity, we assume $d = 2$ when we present the construction algorithm in the following. However, our proposed algorithm could support the construction in higher dimensional spaces with an arbitrary d.

Use of Quad-Tree. Let $\mathcal{S}$ be the set of objects in $[0, 1]^d$ space. We recursively partition the space containing objects into 2^d equal-sized sub-spaces, use a quad-tree T to organize objects based on the partition result, and terminate the partition until every leaf node contains *one and only one* object. The constructed tree has two useful properties, as stated below. Note, given a cube c, $|c|$ indicates its side length and $c[i]$ refers to the smaller coordinate of c in the ith dimension.

PROPERTY 2.1. *The sub-space associated with each node $e \in T$ (denoted as $e.c$) is a standard cube. Here, a cube c is a standard cube iff $\frac{c[i]}{|c|}$ ($\forall i \in [1, d]$) and $\log_2 \frac{1}{|c|}$ are integers.*

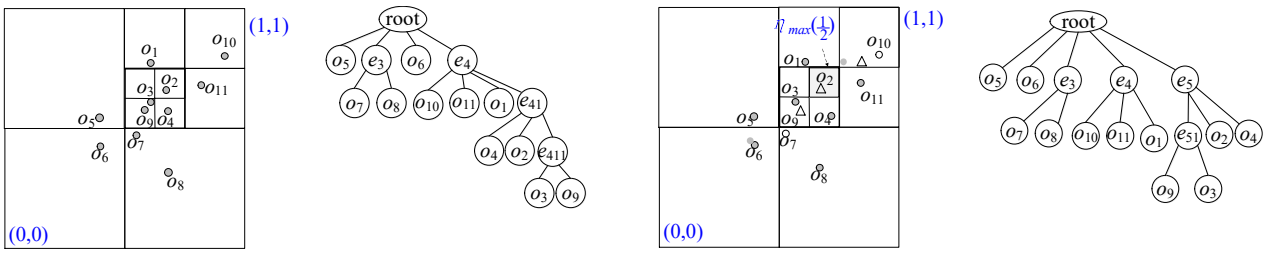
PROPERTY 2.2. *Let $le(o)$ be a leaf node of T that contains an object $o \in \mathcal{S}$ and $c(o)$ be the associated sub-space. $c(o)$ must be the standard cube with maximal side-length that bounds object o exclusively, and the distance from o to its nearest neighbour in $\mathcal{S}$ is bounded by $2\sqrt{d}|c(o)|$, i.e., $D(o) \leq 2\sqrt{d}|c(o)|$.*

Take object o_{11} in Figure 2(a) as an example. It is contained in the leaf node $le(o_{11})$, which is bounded by a standard cube $c(o_{11})$, with $|c(o_{11})| = 2^{-2}$, $c(o_{11})[1] = 0.75$ and $c(o_{11})[2] = 0.5$ (i.e., $\frac{c(o_{11})[1]}{|c(o_{11})|} = 3$, $\frac{c(o_{11})[2]}{|c(o_{11})|} = 2$, and $\log_2 \frac{1}{|c(o_{11})|} = 2$). Based on the quad-tree construction algorithm, $le(o)$ must have at least one non-empty sibling node e' and they share the same parent node e with $|e.c| = 2|c(o)|$. Therefore, there is at least one object (e.g., o' in e') in the search range of $2\sqrt{d}|c(o)|$ centered at o. In other words, we can transform the expensive NN search to range query over quad-tree that is cheaper. For o_{11}, as $|c(o_{11})| = 2^{-2}$, the distance between o_{11} and its NN is bounded by $2\sqrt{2} \cdot 2^{-2} = 2^{-0.5}$.

However, if objects in $\mathcal{S}$ are skewed distributed (e.g., when many objects are densely distributed within a very small subspace), the quad-tree could be tall and the overall running cost will be high. As shown in Figure 2(a), there are in total 11 objects while many of them are located in the top right quadrant. The final quad-tree has the height of five and is very imbalanced.

The Cube-based Partition. After we take a closer look at quad-tree, we understand that the imbalance of quad-tree is resulted from each node e with the distribution of enclosed objects highly skewed, i.e., when e is partitioned into 2^d equal-sized subspaces, most of the enclosed objects (or all the objects in the worst case) are located in one subspace (say e_i) only. In other words, if we could further partition the subspace e_i into two subspaces (say e_i' and e_i'') that enclose roughly same number of objects and replace e_i with e_i' and e_i'' in the original tree, we can effectively reduce the height of the resulted quad-tree. Meanwhile, the partition has to be carefully performed such that the resulted tree still preserves the two properties (i.e., Properties 2.1- 2.2) to ease the search of NN for each object.

In the following, we propose a cube-based partition that is to, for a given node e_i of the quad-tree, find a standard cube c_ξ that is within $e_i.c$ and meanwhile encloses roughly $\frac{|e_i.O|}{2}$ objects in $e_i.O$. Note, $e_i.O$ refers to the set of objects fallen within node e_i. The proposed algorithm is based on a novel concept of maximal median cube, as defined below. Here, η_u in the form of $\langle \eta_u[1], \eta_u[2], \ldots, \eta_u[d] \rangle$



(a) The Quad tree -based Partition (b) The Quad Tree Construction (c) The Qube-based Partition (d) The QC Tree Construction

Figure 2: The NN-Core Construction.

refers to a virtual d−dimensional point, where $\eta_u[j]$ is set to the u-quantile value among objects' coordinates along the jth dimension.

DEFINITION 2. **Maximal Median Cube**. *Given a data set $\mathcal{P}$ and virtual points $\eta_{\frac{1}{2}}$, $\eta_{\frac{1}{4}}$ and $\eta_{\frac{3}{4}}$, the maximal median cube (in short max cube), denoted as $c_{max}(\eta_{\frac{1}{2}})$, refers to a standard cube with maximal side-length that contains $\eta_{\frac{1}{2}}$ but neither $\eta_{\frac{1}{4}}$ nor $\eta_{\frac{3}{4}}$.*

Consider the set of 7 objects contained in e_4 of Figure 2(b)-(c). $\eta_{\frac{1}{4}}$, $\eta_{\frac{1}{2}}$, and $\eta_{\frac{3}{4}}$ points are represented by three triangles. As o_9 has the 2-nd smallest coordinates in both 1st-dimension and 2nd-dimension ($\frac{2}{7}$ is approaching to $\frac{1}{4}$), $\eta_{\frac{1}{4}}$ is identical to o_9. Similarity, $\eta_{\frac{3}{4}}$ is identical to o_2, while $\eta_{\frac{3}{4}}$ is a virtual object with $\eta_{\frac{3}{4}}[1] = o_{11}[1]$ and $\eta_{\frac{3}{4}}[2] = o_1[2]$. Accordingly, the maximal median cube $c_{max}(\eta_{\frac{1}{2}})$ is a standard cube with side-length 2^{-3} that bounds only o_2 (the shaded square in Figure 2(c)).

We now formally explain the partition algorithm. Given a node e_i of quad-tree, we first form $c_{max}(\eta_{\frac{1}{2}})$ based on objects in $e_i.O$ and we then map each object $o_j \in e_i.O$ into a standard cube c_j via the following two mapping rules, dependent on whether o_j is located inside $c_{max}(\eta_{\frac{1}{2}})$. All the generated standard cubes c_js are maintained by a set $C = \{c_1, c_2, \ldots, c_{n'}\}$ with $n' = |e_i.O|$. Note, some cubes in C might be identical.

- Rule (1): If o_j is located outside $c_{max}(\eta_{\frac{1}{2}})$ (i.e., $o_j \notin c_{max}(\eta_{\frac{1}{2}})$), we find a minimal standard cube c that bounds both o_j and $c_{max}(\eta_{\frac{1}{2}})$ and then map o_j into a standard cube c_j that bounds o_j and meanwhile has side-length $\frac{|c|}{2}$.
- Rule (2): Otherwise, we map o_j into $c_{max}(\eta_{\frac{1}{2}})$, i.e., $c_j = c_{max}(\eta_{\frac{1}{2}})$.

We propose the above two mapping rules to achieve two desirable properties. First, any cube in C contains no more than $\frac{|e_i.O|}{2}$ objects, as stated in Lemma 3. Second, we can form c_ξ based on the cubes contained in C, as facilitated by Lemma 4. Specially, we can perform median search to look for the median value s_{me}, considering the side-lengths of all the cubes in C. We then construct a standard cube c_ξ with side-length $\min(2s_{me}, \frac{|e.c|}{2})$ containing $c_{max}(\eta_{\frac{1}{2}})$. The side-length of c_ξ and the fact that c_ξ bounds $c_{max}(\eta_{\frac{1}{2}})$ guarantee that c_ξ must be within $e_i.c$. As stated in Lemma 4, cubes in C with side-length no larger than $\min(2s_{me}, \frac{|e_i.c|}{2})$ are all contained in c_ξ (s_{me} equals to δ in Lemma 4). Together with Lemma 3, the number of objects in c_ξ is approaching $\frac{|e_i.O|}{2}$.

Back to the example in Figure 2(c). The shaded cube that contains object o_2 and has 2^{-3} as the side-length is the $c_{max}(\eta_{\frac{1}{2}})$. We map those seven objects into standard cubes based on the two rules defined above. For example, as o_3 and $c_{max}(\eta_{\frac{1}{2}})$ are bounded by a cube with side-length 2^{-2}, o_3 is mapped into a cube with side-length 2^{-3} based on rule (i); o_2 is mapped into $c_{max}(\eta_{\frac{1}{2}})$ based on rule (ii). Based on the side-length of cubes that these 7 objects are mapped to, $2s_{me}$ is 2^{-2} and $\frac{|e_4.c|}{2} = 2^{-2}$. Accordingly, the side-length of c_ξ is set to 2^{-2} that contains o_2, o_3, o_4, and o_9.

Note, though we are not able to theoretically prove that the number of objects contained in c_ξ is approaching $\frac{|e_i.O|}{2}$, it helps us achieve the goal of controlling the height of the index tree regardless of the distribution of objects (to be detailed next).

LEMMA 3. *Let $e_i.O$ be the set of objects in e_i, $c_{max}(\eta_{\frac{1}{2}})$ be the max cube constructed based on objects in $e_i.O$, $e_i.c$ be the standard cube with minimal side-length that bounds objects in $e_i.O$, and c be any standard cube in the space of $e_i.c$ not covering $c_{max}(\eta_{\frac{1}{2}})$. It is guaranteed that both c and $c_{max}(\eta_{\frac{1}{2}})$ contain* no *more than $\frac{|e_i.O|}{2}$ objects.*

PROOF. See APPENDIX-A □

LEMMA 4. *Given $C_\delta = \{c \in C | |c| \leq \delta\}$, there must exist a standard cube with side length 2δ that bounds all the cubes in C_δ.*

PROOF. See APPENDIX-A □

The Index QC-Tree. Based on cube-based partition, we propose a novel index named *Quad Cube-Tree* (in short QC-Tree) with height bounded by $O(\log(n))$, regardless of the object distribution. Let $\mathcal{S}$ be the set of streaming data in $[0, 1]^2$ space to be organized by a QC-Tree $\mathcal{I}$. Each node $e \in \mathcal{I}$, in the form of $\langle c, n, O, max \rangle$, corresponds to a standard cube $e.c$ and a set of $e.n$ objects $e.O$ fallen within $e.c$ (i.e., $e.n = |e.O|$), with $e.max$ referring to the maximal score of objects in e (i.e., $e.max = max_{o \in e.O} \mathsf{D}(o)$). The reason we introduce $e.max$ will be explained in Section 2.3.

A QC-Tree $\mathcal{I}$ is constructed in the same manner as Quad-Tree. The only difference is that QC-Tree introduces a new node e_5, the fifth child node of e, if the distribution of objects in $e.O$ is skewed. To be more specific, it adopts the number of objects fallen within each quadrant as the metric. Given a node e that has in total $e.n$ objects, if the number of objects associated with any child node e_i exceeds $\frac{e.n}{2}$, the object distribution is considered skewed and the fifth child node e_5 is added to e. Objects in e_i are further partitioned

into two subsets of similar sizes, corresponding to two sub-spaces of $e_i.c$ via *cube-based* partition.

An example is plotted in Figures 2(c) and (d). Initially, following the partition of quad-tree, the root node has four child nodes (i.e., e_1 to e_4). As node e_4 has 7 objects (o_1, o_2, o_3, o_4, o_9, o_{10}, and o_{11}) with $7 > \frac{e.n}{2} = \frac{11}{2}$, we start constructing c_ξ via the *cube-based* partition algorithm. Accordingly, node e_5 is formed which bounds all the objects inside c_ξ (e.g., objects o_2-o_4 and o_9); and node e_4 is updated as $e_4.O$ only includes objects outside of c_ξ.

The NNS Construction. After QC-Tree $\mathcal{I}$ is constructed, we conduct nearest neighbour searches based on Property 2.2. First, for each object $o \in \mathcal{S}$, we submit a range query with query radius $2\sqrt{d}|c_l(o)|$ to look for its NN. Here, given a leaf node $le(o) \in T$ that contains an object $o \in \mathcal{S}$, *L cube* of an object o, denoted as $c_l(o)$, is the standard cube used as the boundary of $le(o)$. As the search process on $\mathcal{I}$ is similar to that on quad-tree, we skip the details. Once NN of every object is found, we construct NNS $\mathcal{N}$. In the event that o and o' are mutual nearest neighbor MNN (i.e., $o = NN(o') \wedge o' = NN(o)$), they only form one pair in NNS $\mathcal{N}$. Last but not least, $e.max$s of different nodes are updated, propagated from leaf nodes to their parent nodes till the root node.

THEOREM 1. *Given a non-leaf node e of QC-Tree, let Child(e) represent all its child nodes. If there is one child node $e_i \in Child(e)$ having more than $\frac{e.n}{2}$ objects, it is guaranteed that all the child nodes of e_i have no more than $\frac{e.n}{2}$ objects, i.e., if $\exists e_i \in Child(e)$, $e_i.n \geq \frac{e.n}{2}$, then $\forall e' \in Child(e_i)$, $e'.n \leq \frac{e.n}{2}$.*

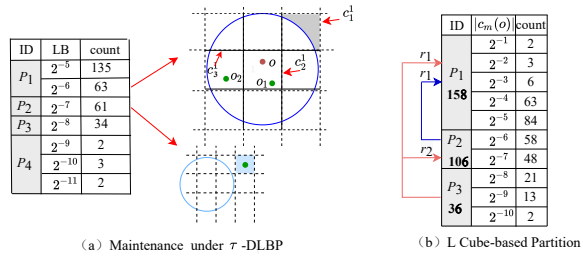
PROOF. See APPENDIX-A □

Discussion. Let e be an internal node of QC-Tree constructed from dataset $\mathcal{S}$ of size n. As stated in Theorem 1, it is guaranteed that each child of e's child contains no more than $\frac{e.n}{2}$ objects. Thus, the height of QC-Tree is bounded by $O(\log n)$. The cost of constructing QC-Tree is bounded by $O(dn \log n)$,i.e., $O(d \log n)$ per object. This property paves the way of efficiently forming and incrementally maintaining $\tau-$DLBP as will be reviewed in Section 3.

2.3 The Maintenance Algorithm

Normally, when an object o_{in} flows into $\mathcal{S}$, we traverse $\mathcal{I}$ to find a leaf node e_f whose space covers o_{in}. Then, e_f will be split into two nodes and the new child node of e_f that bounds o_{in} is a newly generated standard cube. Next, we find the NN of o_{in} and all the existing objects that will take o_{in} as the new NN (i.e., RNN of o_{in}) via searching $\mathcal{I}$, and then update NNS. To be more specific, we only access the nodes e of $\mathcal{I}$ from the root down to the leaf level with their minimum distances to o_{in} no larger than r_d =max($e.max$, $2\sqrt{d}|c_l(o_{in})|$). Recall, $e.max$ captures the maximum score of e's underlying objects. If o_{in} becomes a new NN to an object $o \in e.O$, D(o_{in}, o) shall be smaller than $e.max$. That's why we introduce $e.max$ to each node of $\mathcal{I}$ to facilitate the RNN search. On the other hand, $2\sqrt{d}|c_l(o_{in})|$ bounds the distance from o_{in} to its NN. As the search is straightforward, we skip the details.

When an object o_{exp} expires from $\mathcal{S}$, we remove it from $\mathcal{I}$ and adjust $\mathcal{I}$ if necessary. We then locate, for each o_{exp}'s reverse nearest neighbour o, its nearest neighbour and update NNS accordingly. As the maintenance is simple, we only highlight two points.



(a) Maintenance under τ-DLBP (b) L Cube-based Partition

Figure 3: The Running Example of TNNS and $\tau-$DLBP($\tau = 3$).

Firstly, under object insertion step, when o_{in} reaches a non-leaf node e, if none of e's child nodes contains o_{in}, we construct a new node e' corresponding to a standard cube with side-length $\frac{|e.c|}{2}$ that contains o_{in}, and insert o_{in} into e'. If o_{in} falls within two child nodes (i.e., e_5 and e_i that covers e_5), we insert o_{in} into e_5 to make sure all the objects in e_i that are located within $e_5.c$ are with e_5.

Secondly, as new objects come and existing objects expire, the distribution of streaming data may be changed. Accordingly, object distribution among nodes is also changed, which triggers the reconstruction of certain nodes. Specially, let e be a node of $\mathcal{I}$, e' be its child node, and e'' be a child node of e'. As stated in Theorem 1, $e''.n \leq \frac{e.n}{2}$ when $\mathcal{I}$ is constructed. If $e''.n \geq \frac{3e.n}{4}$, we invoke the construction algorithm to re-organize objects maintained by e. We select $\frac{3e.n}{4}$ as the threshold to reduce the frequency of node re-construction in the premise that the height of $\mathcal{I}$ is not high.

3 SUPPORTING KCP USING A SMALL NUMBER OF NN PAIRS

Although QC-Tree is able to construct and maintain NN pairs for objects in a dynamic setup, it needs to conduct NN search frequently. For example, given an object bounded by a standard cube with large side-length, it needs to scan a large number of nodes in the QC-Tree with cost bounded by $O(n)$. We overcome the above issue via the following facet, that is, the query parameter k is usually a small integer, significantly smaller than the total number of NN pairs in most, if not all, cases. Consequently, it is not necessary to maintain all NN pairs to support KCP. If we obtain enough "high quality" NN pairs, we might be able to use them to answer KCP search. Accordingly, we explore the idea of using a small subset of NN pairs to support the KCP search in this section.

3.1 Grouping Data based on Lower Bound of D(o)

First, we propose the concept of TNNS (short for Threshold-based NN pair Set) to only maintain a small number of NN pairs to address the high maintenance cost of NNS.

DEFINITION 3. **TNNS**. *Let $\mathcal{S}$ be a streaming data set. TNNS $\mathcal{T}$ ($\subset \mathcal{S} \times \mathcal{S}$) consists of all NN pairs with their scores no larger than $g(\tau)$, in the form of NN pairs $(o, NN(o))$. Here, τ ($\ll n$) is an integer, $g(\tau)$ is a score threshold related to τ, and each qualified pair of MNNs is only captured once in TNNS $\mathcal{T}$.*

When a KCP search is submitted, if we can find k pairs with scores bounded by $g(\tau)$ via TNNS, following the idea of Algorithm 1, these k pairs could be used as query result pairs. In order to enable TNNS to

support KCP search even when the above mentioned attempt fails and to incrementally maintain its structure under dynamic data stream environment, we propose a novel partition named τ-DLBP (short for τ-Distance Lower-Bound based Partition) to organize objects, as formally presented in Definition 4. Different from QC-Tree that maintains every object's NN, τ-DLBP only uses one partition to maintain NNs of objects that contribute to TNNS, and uses other partitions to maintain the distance lower bounds of the rest of objects to their NNs. Here, $\mathsf{LB}(P_i)$ ($= 2^u$ with u being a negative integer) refers to the lower bound of scores for objects in partition P_i, i.e., $\forall o_j \in P_i$, $D(o_j) \geq \mathsf{LB}(P_i)$, $|P_i|$ refers to the number of objects in P_i.

DEFINITION 4. *τ−**DLBP**. A τ-DLBP $\mathcal{P}$ groups the streaming data set $\mathcal{S}$ into a group of m partitions $\{P_1, P_2, \ldots, P_m\}$ satisfying:* (1) $LB(P_{m-1}) > g(\tau)$; (2) $\forall i \in (1, m)$, $LB(P_{i-1}) \geq 2LB(P_i)$; *and* (3) $\forall i \in [1, m)$, $|P_i| \geq \sum_{j=i+1}^{m} |P_j|$.

Constraint (1) of τ-DLBP guarantees that all the objects contributing to TNNS are maintained in P_m, though P_m may contain other objects as well. Constraint (2) allows τ-DLBP to use other partitions to maintain objects based on the *lower-bound* of their scores, which will facilitate the process of KCP search with a large k and meanwhile reduce the construction/maintenance cost of τ-DLBP as finding the lower bounds of objects' scores is easier and cheaper, as compared with the finding of objects' exact scores. Constraint (3) further limits the total number of partitions (i.e., m) to $O(\log \frac{|\mathcal{S}|}{\tau})$. In addition, objects with smaller scores (i.e., having shorter distances to their NNs) are contained in smaller partitions with higher granularity, as they have higher chances to contribute to KCP search.

For the setting of $g(\tau)$, let $F(\tau)$ be the τ-th smallest score of objects in $\mathcal{S}$. If $2^{\lceil \log F(\tau) \rceil} = F(\tau)$, $g(\tau) = F(\tau)$; otherwise, $g(\tau)$ is set to $2^{\lceil \log F(\tau) \rceil} - \delta$ with δ (> 0) being a small value approaching 0, which ensures the constraint $\mathsf{LB}(P_{m-1}) > g(\tau)$ is held even when $\mathsf{LB}(P_{m-1}) = 2^{\lceil \log F(\tau) \rceil}$. Figure 3(a) shows an example of τ−DLBP that partitions objects in $\mathcal{S}$ into 4 partitions, based on $\tau = 3$, $F(3) = 2^{-10}$ and $g(\tau) = 2^{\lceil \log F(3) \rceil} = 2^{-10}$. Accordingly, $\mathsf{LB}(P_1)= 2^{-6}$, $\mathsf{LB}(P_2)= 2^{-7}$, and $\mathsf{LB}(P_3)= 2^{-8}$,i.e., $\mathsf{LB}(P_1) \geq 2\mathsf{LB}(P_2)$, and $\mathsf{LB}(P_2) \geq 2\mathsf{LB}(P_3)$. In addition, $|P_1| = 198$(135+63), which is larger than $|P_2| + |P_3| + |P_4|$. $|P_2| = 61$, which is larger than $|P_3| + |P_4|$. Objects with scores smaller than 2^{-8}, in total 7, are contained in P_4.

3.2 The Partition Construction

In this paper, we propose to form τ−DLBP via following three steps, including i) *L cube-based partition*, ii) *score lower-bound calculation*, and iii) *partition refinement*. It firstly forms a partition based on the distribution of L cubes' side-length. Guided by the partition result, it then invokes a group of range queries with relatively *small query radius* to find the NNs of objects that contribute to $T.obj$ and derive the distance lower-bound of the rest of objects to their NNs. It finally forms the partition τ−DLBP. Here, given a leaf node $le(o) \in T$ that contains an object $o \in \mathcal{S}$, *L cube* of an object o, denoted as $c_l(o)$, is the standard cube used as the boundary of $le(o)$, $\mathcal{T}.obj$ refers to the set of objects that contribute to at least one NN pair in $\mathcal{T}$.

L cube-based Partition. The first step is to partition objects $o \in \mathcal{S}$ into different subsets $\{P_1, P_2, \cdots\}$ based on the side-lengths of their L cubes. To be more specific, P_1 contains objects o with $|c_l(o)| \geq m_1$, where m_1 refers to the median of side-length among L Cubes corresponding to objects in $\mathcal{S}$; P_2 contains objects o' with $|c_l(o')| \geq m_2$, where m_2 refers to the median of side-length among L cubes corresponding to objects in $\mathcal{S} - P_1$; and so on. Let's assume that after i partitions, the median of side-length among L Cubes of the remaining objects is bounded by $2^{1+\lceil \log \sqrt{d} \rceil}|c^{2\tau}|$ (denoted as m'_i), where $c^{2\tau}$ refers to the L Cube having the 2τ−th shortest side-length. We then construct partition P_m (i.e., $m = i + 1$) to contain all the objects whose L Cubes have side-length no larger than m'_i and merge the remaining objects (whose L Cubes' side-length is larger than the above threshold m'_i) into the partition P_i. Accordingly, m_i is updated to m'_i. The reason we select median for partitioning is to guarantee $|P_i| \geq \sum_{j=i+1}^{m} |P_j|$ after this L Cube-based partition. After this step, there are in total m partitions, and each partition P_i is associated with a threshold m_i that captures the L cube side length lower bound of objects in P_i, i.e., $\forall o \in P_i$, $|c_l(o)| \in [m_i, m_{i-1})$. In addition, as m'_i is set to $2^{1+\lceil \log \sqrt{d} \rceil}|c^{2\tau}|$, it is guaranteed that we can find τ NN pairs with smallest scores , which will be discussed later.

Consider the examples shown in Figure 3(b). There are in total 300 objects with $\tau = 3$. After the first step of L Cube-based partition, P_1 and P_2 contain 158(=2+3+6+63+84) and 106(=58+48) objects respectively, with $m_1 = 2^{-5}$ and $m_2 = 2^{-7}$. Given $\tau = 3$ and $|c^{2\tau}| = 2^{-10}$, the partition stops as the remaining 36 objects have the side-length of their L Cubes bounded by $2^{1+\lceil \log \sqrt{d} \rceil}|c^{2\tau}| = 2^{-8}$.

Score lower-bound calculation. As the above partition is based on the side-length of objects' L Cubes, we need to compute their real scores (or the lower bound of the scores). Though we can do so via issuing a range query per object o to locate all the objects within $2\sqrt{d}|c_l(o)|$ to o via QC-Tree, we propose a different approach to fully utilize a key property of τ−DLBP after the first step (i.e., $\forall o \in P_i$, $|c_l(o)| \in [m_i, m_{i-1})$).

To be more specific, $\forall o \in P_i$ ($i \in [1, m]$), we submit in total i range queries, denoted as $\cup_{j \in [1,i]} q_j(o, m_j, P_j)$, to locate the objects in partition P_j having distances to o bounded by m_j (e.g., objects in P_1 only issue one range query with radius m_1, objects in P_2 only issue two range queries with radius m_1 and m_2 respectively, and etc). Meanwhile, we maintain, for each object o, $o.NN$ and $o.d$ to capture o's current NN among all the objects located by the above mentioned range queries and the corresponding distance respectively. Initially, $o.NN$ is set to $NULL$ and $o.d = \infty$. Note, $o.NN$ is not only determined by objects retrieved by the i range queries centered at o, as it might be updated when o falls within any range query $q_i(o', m_i, P_i)$ issued by object $o' \in P_l$ ($l > i$), i.e., if o is retrieved by $q_i(o', m_i, P_i)$ and $\mathsf{D}(o, o') < o.d$, then $o.NN = o' \wedge o.d = \mathsf{D}(o, o')$.

THEOREM 2. *Once all the range queries corresponding to objects in $\mathcal{S}$ are processed, $\forall o \in P_i$, if $o.NN$ is NULL, we have $m_i < D(o)$; otherwise, $o.NN = NN(o)$.*

PROOF. See APPENDIX-A □

We want to highlight three unique features of the above mentioned search process based on range queries. Firstly, once all the range queries corresponding to objects in $\mathcal{S}$ are processed, we can find, for each object $o \in \mathcal{S}$, either its real nearest neighbor $NN(o)$ or the lower bound of its score, as stated in Theorem 2. Secondly, the NNs located by above range queries also allow us to construct TNNS. This is because the search radius of all the range queries

is no smaller than m_m ($m_m = 2^{1+\lceil \log \sqrt{d} \rceil}|c^{2\tau}|$). In addition, there are at least 2τ objects whose L Cubes' side-length is no larger than $|c^{2\tau}|$. According to Property 2.2, those objects will have their scores bounded by $2\sqrt{d}|c^{2\tau}|$. As $2^{1+\lceil \log \sqrt{d} \rceil}|c^{2\tau}| \geq 2\sqrt{d}|c^{2\tau}|$, we are able to find all the NN pairs with scores bounded by $2\sqrt{d}|c^{2\tau}|$ and the number is at least τ. Accordingly, we can compute $F(\tau)$ and $g(\tau)$. Thirdly, each object $o \in P_i$ submits i range queries. Given a range query $q(o, m_i, P_i)$, the search region overlaps with maximum 3^d cubes with side-length m_i, each cube contains at most 1 object, and $|P_i| \leq \frac{n}{2^i}$, so the running cost is $O(\sum_{i=1}^{m} i\frac{n}{2^i} + \tau^2)$ (i.e., $O(3^d n + \tau^2)$), which is significantly lower than that is under QC-Tree.

Partition Refinement. The last step is to re-locate objects to the right partitions based on $o.d$, their scores or the lower bounds of the scores, following the same logic used in the first step. The only difference is that L-Cube-based partition is based on the side-length of objects' L Cubes, while the partition refinement is based on $o.d$. To be more specific, we first compute $F(\tau)$. Next, we form the new P_1 to contain objects o with $o.d \geq m_1$ ($= 2^{\lfloor \log m'_1 \rfloor}$), where m'_1 refers to the median of $o.d$ for all the objects o in $\mathcal{S}$; we form P_2 to contain objects o' with $m_1 > o'.d \geq m_2 (= 2^{\lfloor \log m'_2 \rfloor})$, where m'_2 is set to the median of $o.d$ corresponding to objects o in $\mathcal{S} - P_1$, and so on. The reason we use $2^{\lfloor \log m'_i \rfloor}$ instead of m'_i as the distance threshold in this step is to make sure $\mathsf{LB}(P_i)$ (i.e., m_i) is in the form of 2^u. The partition continues until the median of $o.d$ of remaining objects is no larger than $2^{\lceil \log F(\tau) \rceil}$. Assume we have in total i partitions at this moment, we form P_m (i.e., $m = i + 1$) to accommodate all the remaining objects. Accordingly, we set $g(\tau)$ based on $F(\tau)$, and then construct TNNS $\mathcal{T}$. Note, partitions P_is with $i \in [1, m-1]$ set m_i to the lower bounds of the scores corresponding to the objects within the partitions, while P_m sets m_m to $g(\tau)$.

Back to the example in Figure 3(a). Recall that we have $\tau = 3$, $F(3) = 2^{-10}$. After P_1, P_2 and P_3 are formed, the remaining 7 objects have their corresponding median of $o.d$ smaller than 2^{-10}. Accordingly, $g(\tau)$ is set to 2^{-10}. The refinement of $\tau-$DLBP can be terminated and those 7 objects form the last partition P_4. NN pairs with scores bounded by $g(\tau)$, in total 3, form $\mathcal{T}$.

Once $\tau-$DLBP is constructed, for each partition P_i, we maintain another data structure C_i that refers to the set of standard cubes with side-length m_i that bound at least one object o in current partition or any of the subsequent partitions $\cup_{l=i}^{m} P_l$, i.e., $\forall o \in \cup_{l=i}^{m} P_l$, $\exists c \in C_i$ such that c is a standard cube that bounds o and has $|c| = m_i$. Each cube $c_j \in C_i$ is in the form of $\langle n_j^i, O_j^i \rangle$, where n_j^i records the total number of objects in subsequent partitions $\cup_{l=i+1}^{m} P_l$ that fall within the standard cube c_j, and O_j^i refers to the set of objects in P_i that are within c_j. As P_m does not have any subsequent partition, C_m maintains cubes with side-length m_m to maintain objects contained in P_m, i.e., each cube $c_m^j \in C_m$ only captures O_j^m but not n_j^m.

In Figure 3(a), c_1^1 and c_3^1, two standard cubes with side-length 2^{-6}, are both associated with P_1. c_1^1 contains zero object in P_1 and one object in subsequent partitions, and c_3^1 contains object o_2 in P_1 and zero object in subsequent partitions. Accordingly, we have $n_1^1 = 1$, $O_1^1 = \emptyset$, $n_3^1 = 0$, and $O_3^1 = \{o_2\}$. In our detailed implementation, we use a group of m hash tables [11] $\{H_1, H_2, \cdots, H_m\}$ with $\{2|P_1|, 2|P_2|, \cdots, 2|P_m|\}$ buckets to maintain objects in these partitions. Take o_2 in Figure 3(a) as an example. Since o_2 is mapped into a cube $c_1^3 \in C_1$ with side-length 2^{-6}, we use the Z-address of c_1^3 as o_3's key and map o_3 into a bucket $b \in H_1$.

3.3 The Partition Incremental Maintenance

After τ-DLBP and TNNS are constructed, objects in $\mathcal{S}$ might be changed as new objects come and/or existing objects expire. Consequently, it is critical to maintain τ-DLBP and TNNS in a dynamic manner. In the following, we explain how to update τ-DLBP and TNNS when $\mathcal{S}$ is changed.

For newly arrived objects. Given a newly arrived object o, we need to derive its score (or the lower bound of its score) and map o to a proper partition P_i. To achieve this goal, we gradually evaluate partitions from P_1 to P_m by performing two tasks, including a range query and an overlap checking. The range query is to locate objects that might be the nearest neighbor to o and the overlap checking is to determine whether the evaluation of the partitions could be safely terminated, as it does not necessarily require evaluating all m partitions to find the proper partition P_i for object o. Along the evaluation, we may move objects that take the new object o as the nearest neighbor from their original partitions to proper partitions to reflect their reduced scores.

When evaluating a partition P_j ($j < m$), the corresponding range query $q_j(o, m_j, P_j)$ is to locate all the objects in P_j that are within m_j distance to o. $\forall o' \in q_j(o, m_j, P_j)$, if $\mathsf{D}(o, o') < m_j$, it indicates that the new object o becomes the nearest neighbor of o' and o''s score is no longer bounded by m_j. Accordingly, we move o' from P_j to the partition P_u satisfying $m_u \leq \mathsf{D}(o') < m_{u-1}$. Meanwhile, for each cube set $C_v (j \leq v \leq u)$ corresponding to P_v, if there is $c' \in C_v$ that bounds o', we increase $c'.n$ by one; otherwise (i.e., $\forall c' \in C_v$, standard cube c' does not cover o'), we create a new standard cube c' of side-length m_v such that $c'.O = \emptyset$ and $c'.n = 1$. In particular, if $v = u$, o is added into $c'.O$.

Different from the range search, the overlap checking is to check whether the search range of q_j, denoted as $range(o, m_j)$, overlaps with standard cubes in C_j associated with P_j of side-length m_j. As we evaluate partitions from P_1 to P_m (i.e., the index j of a partition increases its value), the side-length m_j of the standard cubes in C_j (that is also the radius m_j of the search range) becomes smaller with the increasing of j, their overlap becomes smaller too. Let P_i be the first partition such that either the overlap is reduced to zero (i.e., $C_i^{q_i} = \emptyset$) or even if the overlap is not empty but none of objects in subsequent partitions actually falls within the overlaps (i.e., $\sum_{c \in C_i^{q_i}} c.n = 0$) As stated in Property 3.1, $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$ guarantees that the distance between any object in the subsequent partitions and object o is longer than m_i. Here, $C_i^{q_i}$ refer to the set of standard cubes in C_i having overlap with $range(o, m_i)$.

Property 3.1. *Let C_i be the set of standard cubes with side length m_i, and $C_i^{q_i}$ refer to cubes in C_i that overlap with the search region $range(o, m_i)$ of q_i, i.e., $C_i^{q_i} = \{c \in C_i | c \cap range(o, m_i) \neq \emptyset\}$. If $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$, $\forall o' \in \cup_{j=i+1}^{m} P_j$, $\mathsf{D}(o, o')$ is larger than $\mathsf{LB}(P_i)(\mathsf{LB}(P_i) = m_i)$.*

Once partition P_i ($i < m$) is located, dependent on the value of $\mathsf{D}(o, o')$, we will find a proper partition to host o. Here, o' refers to the object nearest to o, among those retrieved by the range searches. If $\mathsf{D}(o, o') < m_i$, then o will be inserted into a partition P_j ($j > i$) such that $m_j \leq \mathsf{D}(o, o') < m_{j-1}$. Otherwise (i.e., $\mathsf{D}(o, o') \geq m_i$),

o will be inserted into P_i. As guaranteed by Theorem 3, the above process will insert o into the right partition P_i such that $\mathsf{D}(o)$ is within the range of $[(1+\sqrt{d})m_{i-1}, m_i]$, even though object o' is not necessarily the real nearest neighbor of o (i.e., $\mathsf{D}(o,o') \geq \mathsf{D}(o)$). Meanwhile, the standard cube set C_i of the affected partitions will be updated accordingly. If we cannot find such a partition even after evaluating partitions from P_1 to P_{m-1}, it means $C_i^{q_i} \neq \emptyset$ and $\sum_{c \in C_{m-1}^{q_{m-1}}} c.n > 0$. We insert it into P_m. If the above range query returns at least one object with its new score no larger than $g(\tau)$ or $o.d \leq g(\tau)$, we update TNNS $\mathcal{T}$.

Let's revisit example in Figure 3(a). When processing the newly arrived object o, we first submit a range query $q_1(o, m_1, P_1)$ on P_1 with $m_1 = 2^{-6}$, which returns o_1 and o_2. Accordingly, we have $o.NN = o_1$ and $o.d = \mathsf{D}(o, o_1)$. We next perform the overlap checking. As $\sum_{c \in C_1^{q_1}} c.n \neq 0$, we continue to evaluate partition P_2. We again submit a range query $q_2(o, m_2, P_2)$ on P_2 with $m_2 = 2^{-7}$, which does not return any object. We then perform the overlap checking and find that $\sum_{c \in C_2^{q_2}} c.n = 0$. Accordingly, we terminate the search. As $m_2 < o.d \leq m_1$, o will be inserted into P_2; o_1, whose score has been reduced by o, will be moved from P_1 to P_2. Accordingly, the standard cube c_2^1 belonging to C_1 is updated from $\langle o_1, 0\rangle$ to $\langle \emptyset, 2\rangle$ as it includes two objects of P_2 (i.e., o and o_1). The standard cube C_2 associated with P_2 will be updated too.

THEOREM 3. *Given an object o to be remapped/inserted to a partition, assume partition P_i is the first partition such that $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$ and o is inserted into $P_j (1 < j < m)$. It is guaranteed $m_j \leq \mathsf{D}(o) \leq \mu m_{j-1}$ with μ bounded by $(1+\sqrt{d})$.*

PROOF. See APPENDIX-A □

Along with the updates of $\mathcal{S}$, $F(\tau)$, the τ-th smallest score of NN pairs in $\mathcal{T}$, changes its value too. However, $g(\tau)$ is set to $2^{\lceil \log F(\tau) \rceil}$ (or adjusted by δ) based on $F(\tau)$ at the moment TNNS is constructed. Consequently, $g(\tau)$ might be very different from the current $F(\tau)$. Therefore, if $g(\tau) > 2F(\tau)$, it means $g(\tau)$ is so loose that it should be reduced based on the current $F(\tau)$. We update $g(\tau)$ to $2^{\lceil \log F(\tau) \rceil}$ and remove NN pairs in $\mathcal{T}$ whose scores are above the new threshold $g(\tau)$ from $\mathcal{T}$. We also update the cube set C_m associated with P_m to a group of cubes with side-length $2^{\lceil \log F(\tau) \rceil}$ and map objects in P_m to cubes in C_m. As objects are inserted into different partitions, the local re-partition (to be presented in Section 3.4) might be triggered to adjust the partitions.

For expired objects. When an object $o_{exp} \in P_i (i \in [1, m])$ expires from $\mathcal{S}$, we remove it from P_i. Accordingly, for each cube c corresponding to $P_j (j < i)$ whose occupying space contains o_{exp}, we reduce $c.n$ by one. In particular, if c turns to $c\langle 0, \emptyset\rangle$, we delete c from C_j. Note, after o_{exp} expires from $\mathcal{S}$, it affects its reverse nearest neighbours, as their nearest neighbour o_{exp} is expired. Though the partitions that those objects are located in might be no longer valid, we do not perform the update immediately. As the scores of those objects will be increased, the lazy update approach we adopt will not affect the validity of our search algorithms. However, if $o_{exp} \in \mathcal{T}.obj$, we further update TNNS. For each of o_{exp}'s reverse nearest neighbour o in $\mathcal{T}.obj$, i.e., $o \in (RNN(o_{exp}) \cap \mathcal{T}.obj)$, we submit a range query with radius $g(\tau)$ on P_m. If the query result set is not empty, we form a new NN pair based on o and its new nearest neighbour and insert it into $\mathcal{T}$.

3.4 The Local Re-Partition

After τ-DLBP is constructed, the arrival of new objects and/or expiry of existing objects change the partitions. Local re-partition of τ-DLBP is triggered to further adjust the partitions iff i) $|\mathcal{T}| < \frac{\tau}{2}$; or ii) $\exists i \in [1, m-1]$ such that $|P_i| \leq \sum_{j=i+1}^{m} |P_j|$; or iii)$|\mathcal{T}.obj| < \frac{|P_m|}{3}$.

Local Re-Partition Under Condition i). We first focus on condition i) $|\mathcal{T}| < \frac{\tau}{2}$, which occurs after some objects expire from $\mathcal{T}.obj$. As $\tau-$DLBP uses partition P_m to maintain objects with smallest scores (or score lower-bound), we first attempt to use objects in $(P_m - \mathcal{T}.obj)$ to re-form $\mathcal{T}$. If we can find at least τ NN pairs with scores smaller than $\mathsf{LB}(P_{m-1})$, there is no need to search other partitions as scores of objects in other partitions are all no smaller than $\mathsf{LB}(P_{m-1})$. Otherwise, we have to move some objects in other partitions as they will contribute to $\mathcal{T}$, as detailed below.

According to the properties of τ-DLBP,i.e., $\mathsf{LB}(P_i) \geq 2\mathsf{LB}(P_{i+1})$, we always consider objects in P_m and the partition right before P_m. To ease our presentation, we use i to capture the index corresponding to the partition that is currently right before P_m, and it is initially set to $m-1$. Using the lower bound $\mathsf{LB}(P_{i-1})$ associated with P_{i-1} (i.e., initially P_{m-2}) as the threshold, objects in $P_m \cup P_i$ are partitioned into two sets, the one $S_<$ having all the objects whose scores are smaller than $\mathsf{LB}(P_{i-1})$ and the other $S_\geq$ having the remaining objects. Accordingly, objects in $S_<$ are inserted into P_m and objects in $S_\geq$ are inserted into P_{i-1}. Please note that after this step, partition P_i is empty and partition P_{i-1} becomes the new partition that is right before P_m. We can continue the above process by setting i to $(i-1)$ until we can find at least τ objects with the smallest scores, which will be detailed in the next paragraph.

We now explain how to find the real scores for objects in $P_m \cup P_i$ when we evaluate partition P_i that is right before P_m. We first obtain the local-L Cubes for all the objects in $P_m \cup P_i$. A L Cube c_o of o is clustered as a local L Cube if the L Cube is constructed based on a subset $S' \subset \mathcal{S}$. We will explain how to form local L Cubes for objects in S' later. Here, $\forall o \in S'$, its local-L Cube c_o might be different from its exact L Cube $c_l(o)$, as $|c_o| \geq |c_l(o)|$. However, side-lengths of these local-L Cubes could be used to evaluate the upper-bounds of their scores for objects in S' and hence allow us to efficiently re-construct TNNS.

Let n_{lmc} refer to the number of objects whose local-L Cubes have side-length smaller than $\frac{m_{i-1}}{2\sqrt{d}}$. If $n_{lmc} \geq 2\tau$, the number of NN pairs with scores bounded by m_{i-1} must be at least τ (as $\mathsf{D}(o) \leq 2\sqrt{d}|c_l(o)| \leq 2\sqrt{d}|c_o| < m_{i-1}$). Accordingly, we can complete local re-partition after accessing objects in P_i without accessing P_{i-1}. We adopt $\tau-$DLBP construction algorithm presented in Section 3.2 to partition objects in $P_m \cup P_i$ into a group of new partitions.

Otherwise ($n_{lmc} < 2\tau$), we try to adopt range searches to locate NNs of objects in $P_m \cup P_i$ as the side-length of their local-L Cubes might be too loose. We first map objects in $P_m \cup P_i$ into a group of cubes with side-length m_{i-1}. Next, $\forall o \in P_i$, we submit a range query $q(o, m_{i-1} - \delta, P_m \cup P_i)$ to find set of objects R_o in $P_m \cup P_i$ with distances to object o smaller than m_{i-1} (δ is a very small real number approaching 0, initially defined in Section 3.1). If $R_o = \emptyset$, the distance from o to its NN is larger than $(m_{i-1} - \delta)$ and o is then

inserted into P_{i-1}. Otherwise (i.e., $R_o \neq \emptyset$), R_q must contain the NN of o with $\mathsf{D}(o) < m_{i-1}$ and o is inserted into P_m.

After relocating objects to proper partitions, if there are at least τ NN pairs with scores smaller than m_{i-1} in P_m, we execute *partition refinement* step presented in Section 3.2 to construct a group of new partitions based on objects' scores (for objects in P_m), and then re-form $\mathcal{T}$ and update $g(\tau)$.

Note, as scores of objects in the partitions from P_1 to P_{i-1} are all no smaller than m_{i-1}, all NN pairs with scores smaller than m_{i-1} must be could found after above mentioned range searches, and we can terminate the local re-partition. On the other hand, if the number of NN pairs formed by objects in P_m with scores no larger than m_{i-1} is smaller than τ, we set $i = i - 1$ and repeat the above process again.

We want to highlight that when objects expire, the scores of some objects might become larger. Consequently, lower bounds associated with each partition might be very loose (e.g., the real lower bounds are much larger than the current lower bounds). It implies that we might not be able to terminate the above search even after searching all the partitions. For such an extreme case, we can re-construct $\tau-$DLBP from scratch based on the $\tau-$DLBP construction algorithm presented in Section 3.2.

We now explain how to construct and maintain local L Cubes of objects under data stream. In order to reduce maintenance cost, we use a QC-Tree liked index $\mathcal{I}'$ to maintain objects in S'. Here, an object $o \in \mathcal{S}$ is included into S' iff i) o has participated in at least one local re-partition of $\tau-$DLBP with $o.t$ capturing the last time o participates in a local re-partition and ii) the number of updates(object insertions and deletions) to $\mathcal{S}$ since $o.t$ is less than $2n$ times. In other words, assume a local re-partition of $\tau-$DLBP happens at timestamp t, which involves an object o. Accordingly, o is inserted into $\mathcal{I}'$ at t. Thereafter, S has lots of updates, and the total number of updates reaches $2n$ at t'. If o does not participate in any local re-partition after t, it is deleted from $\mathcal{I}'$ at t'. Each object $o \in \mathcal{S}'$ is associated with a tuple in the form of $\langle t, e_f \rangle$. $o.t$ refers to the last time o participates in a local re-partition, and $o.e_f$ refers to the corresponding leaf node in $\mathcal{I}'$ that bounds o. $\mathcal{I}'$ is updated whenever (i) an object $o \in \mathcal{I}'$ expires from $\mathcal{S}$ or deleted from $\mathcal{S}'$ as explained above; or (ii) a new local re-partition is invoked and objects newly participated into this local re-partition are inserted into $\mathcal{I}'$.

Local Re-Partition Under Other Conditions. For condition ii) $|P_i| < \sum_{j=i+1}^{m} |P_j|$, we can consider objects in $\cup_{j=i}^{m} P_j$ as a dataset and invoke *partition refinement* step introduced in Section 3.2 to perform local re-partition. For case iii) $|\mathcal{T}.obj| < \frac{|P_m|}{3}$, i.e., many objects with scores(or score lower-bounds) ranging from m_m to $g(\tau)$ are contained in P_m, we split objects in P_m into P_m and P_{m+1}, again following the same logic as *partition refinement* step introduced in Section 3.2. Accordingly, we form cube sets C_m and C_{m+1} for maintaining objects in P_m and P_{m+1} respectively.

Discussion. In an extreme case, the local re-partition may incur a relatively high cost, if the top-τ NN pairs with smallest scores keep changing. For example, many objects of $\mathcal{T}$ are expired which triggers the local re-partition because of condition (i). Accordingly, $F(\tau)$ and $g(\tau)$ are increased and objects from other partitions (e.g., P_{m-1}) will be merged into partition P_m. As a result, P_m may become much larger, which in turn triggers the local re-partition because of condition (iii) and partition P_m is split into two partitions. If objects in the newly formed $\mathcal{T}$ are expired again, P_m again requires expansion by merging objects from other partitions. The merge and split of P_m may happen frequently. Though this refers to an extreme case with a very low occurrence probability, we do provide a remedial solution by maintaining extra information related to NN pairs whose scores are within the range of $(0.5g(\tau), g(\tau)]$ to mitigate the high update cost. Please see the details in APPENDIX-B.

3.5 KCP Search Under TNNS and $\tau-$DLBP

When a KCP search q is submitted, if we can form k object pairs with smallest scores smaller than $g(\tau)$ by considering objects in TNNS, following the logic presented in Algorithm 1, these k pairs could be used as query result pairs, and the search could be terminated. Otherwise, we scan objects in other partitions to find k object pairs with the smallest scores, following the same logic used to perform local re-partition under condition (i). For example, if existing k pairs in P_m with scores no larger than $\mathsf{LB}(P_{m-1})$, those k pairs form the answer set; if we can find k pairs in $P_m \cup P_{m-1}$ with scores no larger than $\mathsf{LB}(P_{m-2})$, those k pairs for the answer set, and etc. As the algorithm is almost the same as local re-partition under condition i), we skip the details.

In the following, we will briefly discuss the cost of supporting KCP search under $\tau-$DLBP. The $\tau-$DLBP construction involves L Cube-based partition, score lower-bound calculation and refinement. For the first step, for a given $\mathcal{S}$, as each object is partitioned at most $O(\log n)$ times, the cost of constructing L Cubes is bounded by $O(dn \log n)$. As we use quantile search based algorithm to form the partition, the cost of L Cube-based partition is bounded by $O(n + dn \log n)$. The running cost of the second step is $O(3^d n + \tau^2)$ as discussed in Section 3.2. The cost of the last refinement step is $O(n)$. Therefore, the overall running cost of $\tau-$DLBP construction is bounded by $O(n3^d + dn \log n + \tau^2)$. For the incremental maintenance of $\tau-$DLBP, as the number of partitions is bounded by $O(\log \frac{n}{\tau})$, we can prove that the running cost of processing each update is bounded by $O((d + 3^d) \log \frac{n}{\tau} + \tau)$. We also can prove that the cost of supporting KCP search is $O(3^d n_k + k^2 + dn_k \log n_k)$ with n_k being the number of objects that are evaluated during the search. Its value is excepted to $\sum_{j=i}^{j=m} |P_j|$ with j being the minimal partition ID satisfying $\sum_{j=i}^{j=m} |P_j| > 2k$(The details is shown in APPENDIX-C).

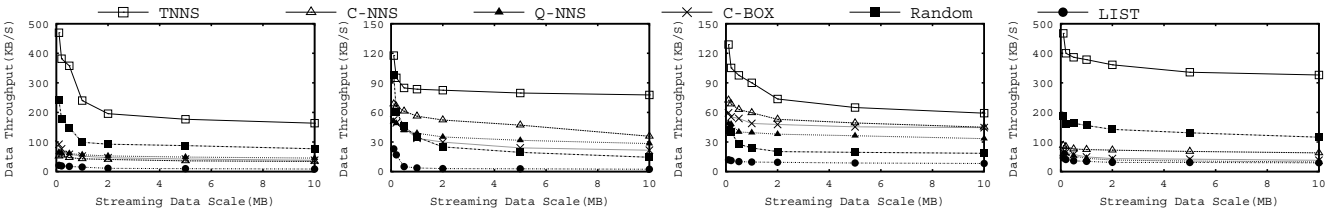
Based on the above analysis, we understand that parameter τ provides a trade-off between the search efficiency of KCP search and the update efficiency of $\tau-$DLBP. A large τ can effectively reduce the search cost, while it increases the $\tau-$DLBP maintenance cost. In our implementation, we set τ to 100. However, its value can be easily adjusted to cater for the needs of different applications.

4 PERFORMANCE EVALUATION

In this section, we conduct extensive experiments to demonstrate the efficiency of proposed techniques. In the following, we first explain the experiment settings and then report our findings.

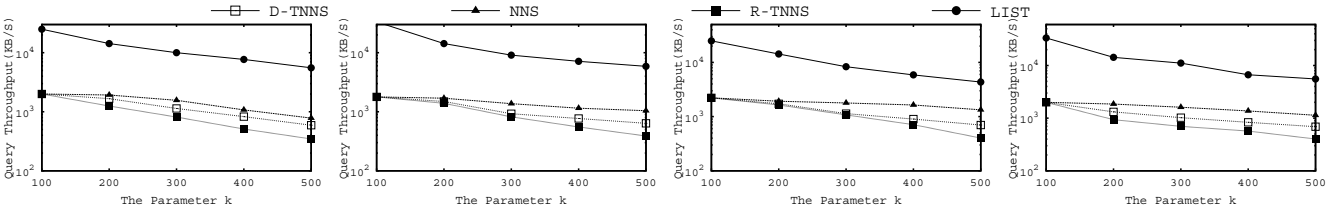
4.1 Experiment Settings

Datasets. In total, four datasets are used in our experiments, including two real datasets, namely TRIP and STOCK, and two synthetic datasets named NORMAL and UNIFORM. TRIP contains 1GB taxi



(a) Stock (b) Trip (c) Uniform (d) Normal

Figure 4: Construction cost comparison under different n



(a) Stock (b) Trip (c) Uniform (d) Normal

Figure 5: KCP Search cost of different algorithms (n = 1MB)

trip records from NYC, collected in a duration of 6 months. We cleaned this dataset by only considering pick-up/drop-off points and travel distance of each trip. Objects are flowed into $\mathcal{S}$ at the moment their travelling plans are submitted to the system, and objects are expired from $\mathcal{S}$ until their associated pick-up time p_t. STOCK refers to a set of 1GB stock transactions corresponding to 2, 300 stocks from Shanghai/Shenzhen Stock Exchange in a duration of 24 months. We cleaned the dataset by only considering volume and price. Objects are flowed into $\mathcal{S}$ based on the order relationship among their associated transaction time, expired from $\mathcal{S}$ after another $|\mathcal{S}|$ objects flow into $\mathcal{S}$ with $|\mathcal{S}|$ being the number of objects in $\mathcal{S}$. STOCK could be regarded as a "special case" of turnstile data stream, where objects' arrival/expiry follow the same order. Compared with other datasets, it can further evaluate the impact of objects' arrival/expiry order to algorithm performance. NORMAL and UNIFORM contain 1GB objects whose dimensionality d could be changed from 2 to 8, with 4 being the default. Objects in these two datasets follow uniform distribution and normal distribution respectively and they arrive in /expire from $\mathcal{S}$ in a random order.

Table 2: Parameter Settings.

Parameter	value
n	100KB, 200KB, 500KB, **1MB**, 2MB, 5MB, 10MB
k	10, 20, 50, **100**, 200, 500, 1000
s	0.01%, 0.05%, **0.1%**, 0.5%, 1%, 5%, 10% ($\times n$)

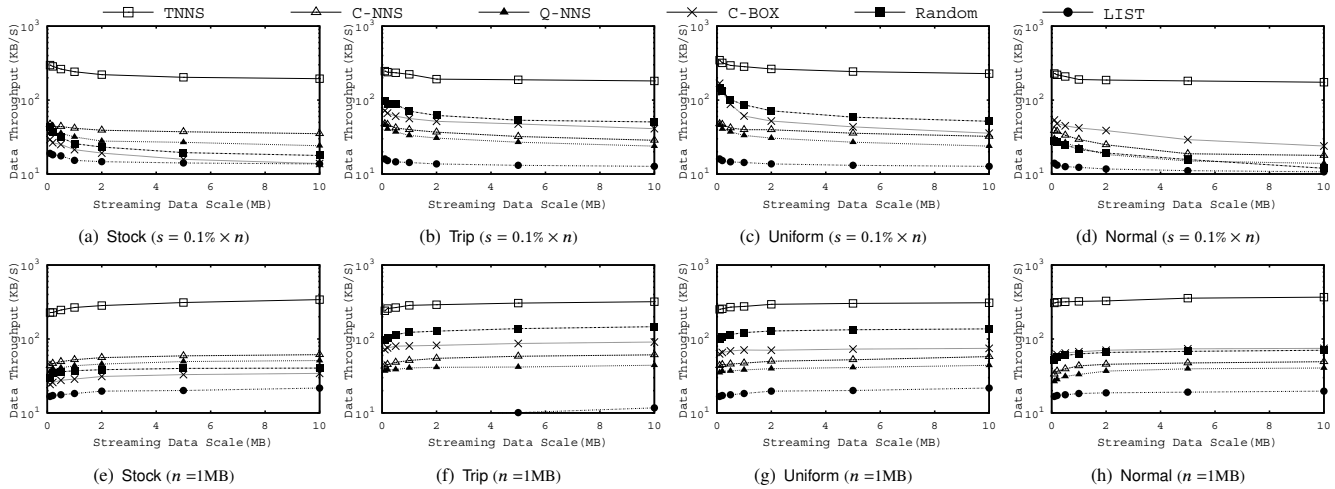
Parameters. We evaluate the performance of different algorithms under three parameters, including n, k, and s. Here, n refers to size of streaming data set $\mathcal{S}$, i.e., the scale of streaming data set; k is the input of KCP search; and s refers to the number of updates per time, i.e., the speed of data stream. The parameter settings are listed in Table 2 with the default values bolded.

Performance Metrics. We consider *data throughput* d_t, *query throughput* q_t and *memory consumption* mc as the main performance metrics. d_t refers to the average number of updates per sec algorithms could process, which indicates how fast the algorithms respond to updates; q_t refers to the average number of queries per sec algorithms could process, which indicates how fast the algorithms respond to KCP search; mc refers to the average memory size consumed by different algorithms. Note, our experiments measure the throughput needed to process each dataset without any data arrival latency. That is, each implementation ingests the streams sequentially and there is zero waiting between successive tuples.

Competitors. This is the first attempt to support KCP search over turnstile stream model. We adapt five baselines with *major* extensions as competitors, including Random [10], C-Box [8], C-NNS, Q-NNS and LIST [12]. Random can support 1CP as it guarantees to have the closest pair in the last partition. To extend Random to support KCP search ($k > 1$), we assume Random uses the last partition to maintain at least 100 NN pairs with the smallest scores. In other words, Random borrows the algorithm from TNNS to support KCP search but adopts a different approach to partition the streaming data. C-Box supports 1CP search via maintaining all mutual nearest neighbours. To enable it to support KCP search, we maintain all NN pairs instead. Note C-Box cannot support KCP search via maintaining a sub set of NN pairs, as it organizes objects based on objects' position relationship instead of distances. Q-NNS and C-NNS use a quad-tree and a QC-Tree to maintain all NN pairs respectively. LIST can support KCP search under sliding window model. It uses a group of d inverted-lists to maintain streaming data. As we cannot use domination relationship among pairs to filter useless ones, we follow the existing work [1] to use a candidate set to maintain $n'(< n)$ object pairs with smallest scores instead. All algorithms are implemented with C++, and all the experiments are conducted on 622R CPU with 256GB memory, running in Win 10.

4.2 Comparison between TNNS and its competitors

Construction Cost Comparison. In our first set of experiments, we evaluate the cost of index construction of all the algorithms under



(a) Stock ($s = 0.1\% \times n$) (b) Trip ($s = 0.1\% \times n$) (c) Uniform ($s = 0.1\% \times n$) (d) Normal ($s = 0.1\% \times n$)

(e) Stock (n =1MB) (f) Trip (n =1MB) (g) Uniform (n =1MB) (h) Normal (n =1MB)

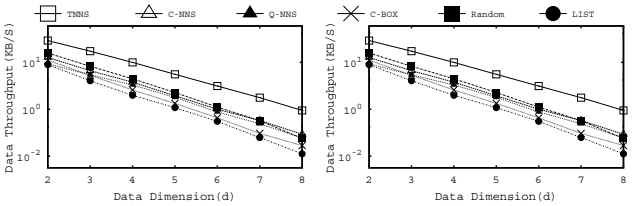
Figure 6: Data throughput comparison of different algorithms under different datasets

different datasets. The parameter n ranges from 100KB to 10MB and τ is set to 100, with results depicted in Figure 4.

As compared with Q-NNS and C-NNS, TNNS construction is more efficient. Under STOCK, TNNS helps to enhance the speed of index construction, ranging from 157% under $n = 100KB$ to 500% under $n = 10MB$. The reason behind is that, TNNS needs not to find all NN pairs, which significantly reduces the total running cost. C-NNS runs faster than Q-NNS. The reason is, under C-NNS, each object could be partitioned into a leaf node exclusively in $O(\log n)$ time, regardless of the objects distribution; while Q-NNS does not have this desirable feature. As a result, the height of QC Tree (used by C-NNS) is fixed at $O(\log n)$ while that of Quad-Tree (used by Q-NNS) could be much larger if the distribution of objects is very skewed. This forces Q-NNS to spend more time in processing range queries when constructing NNS.

TNNS is more efficient than C-Box, Random and LIST too. TNNS could efficiently partition objects based on side-length of objects' L-Cubes; and it only needs to find NNs for a small number of objects. By contrast, Random does not always find suitable *pivot* objects for partitioning, hence objects need to be partitioned many times, which significantly increases the overall construction cost. C-Box has to find NNs for all the objects. For LIST, when constructing the candidate set, as objects in $\mathcal{S}$ have to access multitudes of elements in each inverted-list, the overall constructing cost is high. We also find that TNNS is stable, as it is not sensitive to datasets. By contrast, Random is not stable, as it performs better than C-NNS, C-Box and LIST under TRIP and NORMAL, but performs worse than C-NNS and C-Box under STOCK and UNIFORM. This is because Random randomly selects *pivot* points for partitioning, and the quality of pivots directly affects the partitioning.

KCP search Cost Comparison. In our second set of experiments, we evaluate the cost of KCP search using different algorithms in static dataset. Algorithm NNS (defined in Algorithm 1) uses all NN pairs for supporting KCP search, used by C-Box, Q-NNS, and C-NNS. Algorithm LIST supports KCP search via a set of n' ($k < n' < n$) pairs with lowest scores, used by LIST. Algorithms D-TNNS and R-TNNS use TNNS for supporting query processing, with the former



(a) Uniform (n =1MB and $s = 0.1\% \times n$) (b) Normal (n =1MB and $s = 0.1\% \times n$)

Figure 7: Data throughput comparison of different algorithms under different d using synthetic datasets.

organizing objects based on $\tau-$DLBP and the latter (used by Random) organizing objects randomly.

No surprise that LIST performs the best as it can return query results directly, though it cannot support KCP search when $k > n'$. Besides it, NNS performs better than others as it can fix query result pairs via accessing objects that form k NN pairs with smallest scores. D-TNNS also achieves comparable performance, as its query throughput is about 75% of NNS. This is because it can construct a suitable partition for most objects and the partition size is proper. Thus, even when D-TNNS cannot find enough pairs based on $\mathcal{T}$, the cost of searching other partitions is not high. By contrast, R-TNNS cannot always provide a suitable partition for supporting KCP search. When the algorithm has to search other partitions, the running cost may be high, i.e., it only achieves on average half query throughput of D-TNNS. This well demonstrates the strength of $\tau-$DLBP.

Data throughput Comparison. In our third set of experiments, we compare the data throughput of different algorithms and report the results in Figure 6.

Firstly, we report the data throughput of all the algorithms under different n values in Figures 6(a)-6(d). We can observe that TNNS consistently outperforms all the competitors, for all four datasets. For example, the data throughput of TNNS on average is 300% of C-NNS, 350% of Q-NNS, 450% of C-BOX, 500% of Random, and 2000% of LIST respectively. As compared with C-NNS, Q-NNS and C-Box, TNNS does not need to maintain all NN pairs. As compared with Random, it is guaranteed that the partition count in TNNS is bounded by $O(\log \frac{n}{\tau})$ in the worst cases. We also find that C-NNS

Table 3: Average index size comparison of different algorithms (n = 1MB and τ =100)

2	STOCK	TRIP	NORMAL							UNIFORM						
	d=2	d=3	d=2	d=3	d=4	d=5	d=6	d=7	d=8	d=2	d=3	d=4	d=5	d=6	d=7	d=8
TNNS	49.0	73.2	46.5	74.8	92.1	123.7	144.5	169.3	191.8	41.1	67.4	91.1	117.5	140.8	166.3	190.1
Q-NNS	51.4	77.0	48.5	79.2	105.1	131.4	154.5	178.2	204.1	45.3	72.2	98.9	126.3	151.2	177.4	203.4
C-NNS	53.0	81.0	52.5	82.7	109.2	137.4	165.5	183.6	211.8	48.1	75.7	102.4	130.5	158.2	185.1	214.4
Random	42.0	64.0	43.5	65.1	76.1	98.8	119.4	140.9	162.3	40.1	62.7	84.8	105.1	130.2	153.1	176.4
C-Box	51.6	72.0	49.2	76.9	103.1	128.5	152.3	180.3	206.1	50.6	75.4	101.3	126.5	153.4	179.2	204.3
LIST	56.3	77.0	55.2	77.9	118.8	139.5	166.1	191.2	213.4	53.9	76.8	117.4	135.6	165.9	190.7	212.5

performs better than Q-NNS. The reason is C-NNS uses QC Tree to maintain NN pairs whose height is bounded by $O(\log n)$, while Q-NNS uses quad-tree to maintain NN pairs which may be very tall in many cases, resulting in expensive running cost of processing updates and NN search. In addition, we observe that TNNS is stable with its data throughput not sensitive to data distribution. In contrast, Random is efficient under TRIP and NORMAL but not STOCK and UNIFORM. This is because Random cannot always properly partition the streaming dataset, especially under datasets with skewed distribution. The count of resulted partitions might be too small or too large. When there are too many partitions, each newly arrived object has to access many partitions; on the other hand when there are too few partitions, the cost of local re-partition may be high. Both cases reduce the data throughput.

Secondly, we report the running time of different algorithms under different s in Figures 6(e)-6(h). It is observed that as s increases its value, all the algorithms have higher throughput and TNNS again performs the best. The reason is the larger the s, the lower the cost of processing expiration objects. For example, let $p(o, o')$ be a NN pair. If o and o' both expire from $\mathcal{S}$ at the same time, we need not to re-search their NNs.

Last but not the least, we report the data throughput of all the algorithms under different d in Figure 7. We observe that the data throughput of all the algorithms drops with the increasing of d, and TNNS outperforms all the competitors again. One important reason is, in most cases, the higher the d, the smaller the score difference among objects, and the fewer the partitions we should maintain. Owing to the properties of $\tau-$DLBP, newly arrived objects are inserted into a partition after accessing a constant number of partitions. By contrast, the others all have to spend expensive cost in processing newly arrived objects and hence they are sensitive to d.

Memory Cost Comparison. In our last set of experiments, we compare the memory cost of all the algorithms. As depicted in Table 3, the memory cost difference among these six algorithms is small. All these six algorithms have to spend extra space in maintaining streaming data. For example, TNNS and Random use a group of hash tables to maintain objects in each partition; Q-NNS,C-NNS and C-Box have to maintain all NN pairs; and LIST has to maintain n' pairs having the lowest scores.

5 RELATED WORKS

KCP search is a fundamental yet challenging problem that has been studied for over thirty years. Existing efforts study the KCP search under different data dimensions and various data environments. In the following, we only review the existing works that are relevant to our study, including i) tree-based algorithms; ii) partition-based algorithms; and iii) other types of algorithms.

Tree-based Algorithm. Many *tree-based approaches* use a tree-based structure to maintain every object's NN. When objects are inserted into or deleted from the dataset, both the index and NNs of relevant objects are updated. A tree-based structure was proposed in [13], which uses $O(n \log^d n)$ space and runs in $O(\log \log n \log^d n)$ amortized time per update to support 1CP search. An enhanced index was proposed in [14] that uses $O(n)$ space and runs in $O(\log \log n \log^d n)$ ($d \geq 3$) amortized time per update to support 1CP search, e.g., the running cost is $O(\log^2 n)$ when $d = 2$. The first deterministic data structure that maintains the closest pair in $O(\log n)$ time per update was presented in [8]. Different from previous approaches that need to maintain all NN pairs, it only maintains mutual nearest neighbour pairs. Unfortunately, its computational complexity has a large hidden constant. For example, the cost of processing one insertion and that of processing a deletion is $O(dN_d \log d \log n)$ and $O(dN_d M_d \log d \log n)$ respectively. Both N_d and M_d are large (e.g., $N_d \geq 2401$ and $M_d \geq 8281$ when $d = 2$).

Partition-based Algorithm. A partition-based algorithm is proposed in [10] to support continuous 1CP search, based on a randomized data structure named *sparse partition*. During initialization, the set S_1 is set to $\mathcal{S}$. Then, the algorithm randomly selects an object $o \in \mathcal{S}$ as the pivot, form the set S_1' based on $o.d$ following a group of partition rules. For example, $\forall$o'$\in S_1$, if $\mathsf{D}(o', S_1) > \frac{o.d}{3}$, o' is inserted into S_1'. After S_1' is formed, the algorithm set S_2 to $S_1 - S_1'$, and then split S_2 following the logic of partitioning S_1. Here, $o.d$ refers to the distance between o and its NN. The above operation is repeated m times until the subset S_m includes only two objects. When an object is inserted into $\mathcal{S}$, the algorithm evaluates whether it contributes to 1CP via accessing these m partitions with cost bounded by $O(3^d m)$. In ideal cases, m is excepted to be $O(\log n)$ and the cost of processing an object is $O(3^d \log n)$. However, its performance heavily depends on the selected *pivot* objects. In the worst case, m might be approaching n, when the running time of processing one insertion/deletion is $O(n)$. [15] proposed a parallel batch-dynamic data structure for 1CP search via using the data structure proposed in [10] to support batches of insertions and deletions in parallel.

Other Algorithms. Instead of maintaining NN of objects, [1] uses a buffer to maintain $n'(n' \leq n)$ pairs having the smallest scores. It is the first dynamic algorithm with a running cost of $O(\sqrt{n} \log n)$ per update and space cost of $O(n)$. [12] studied the problem of KCP search over sliding window via the domination relationships among pairs, a unique property under sliding window, to enhance algorithm performance. However, it can not be used under turnstile stream, since it cannot use domination relationship to delete any pair. [16] proposed a kinetic and dynamic data structure for maintaining the closest pair in a set of moving points in the plane. [17] and [18] solved dynamic closest pair problem by supporting arbitrary distance functions respectively.

6 CONCLUSION

In this paper, we propose a novel concept named NNS for supporting KCP search over stream data. It is able to return the query result pairs via accessing $O(k)$ objects. Furthermore, we propose TNNS and a group of algorithms for supporting KCP search under TNNS. We have conducted extensive experiments to evaluate the performance of our proposed algorithms on several datasets with different distributions. The results demonstrate the superior performance of our proposed algorithms. In the near future, we would like to study approximate KCP search under data stream.

REFERENCES

[1] Michiel H. M. Smid. Closest-point problems in computational geometry. In *Handbook of Computational Geometry*, pages 877–935. North Holland / Elsevier, 2000.

[2] Timothy M. Chan. Dynamic generalized closest pair: Revisiting eppstein's technique. In *3rd Symposium on Simplicity in Algorithms, SOSA 2020, Salt Lake City, UT, USA, January 6-7, 2020*, pages 33–37. SIAM, 2020.

[3] Sanguthevar Rajasekaran, Subrata Saha, and Xingyu Cai. Novel exact and approximate algorithms for the closest pair problem. In *2017 IEEE International Conference on Data Mining, ICDM 2017, New Orleans, LA, USA, November 18-21, 2017*, pages 1045–1050. IEEE Computer Society, 2017.

[4] Antonio Corral, Yannis Manolopoulos, Yannis Theodoridis, and Michael Vassilakopoulos. Closest pair queries in spatial databases. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, May 16-18, 2000, Dallas, Texas, USA*, pages 189–200. ACM, 2000.

[5] John Kallaugher and Eric Price. Separations and equivalences between turnstile streaming and linear sketching. In *Proccedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing, STOC 2020, Chicago, IL, USA, June 22-26, 2020*, pages 1223–1236. ACM, 2020.

[6] Ge Luo, Lu Wang, Ke Yi, and Graham Cormode. Quantiles over data streams: experimental comparisons, new analyses, and further improvements. *VLDB J.*, 25(4):449–472, 2016.

[7] Kasper Green Larsen, Jelani Nelson, and Huy L. Nguyên. Time lower bounds for nonadaptive turnstile streaming algorithms. In *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, pages 803–812. ACM, 2015.

[8] Sergei Bespamyatnikh. An optimal algorithm for closest-pair maintenance. *Discret. Comput. Geom.*, 19(2):175–195, 1998.

[9] Christian Schwarz, Michiel H. M. Smid, and Jack Snoeyink. An optimal algorithm for the on-line closest-pair problem. *Algorithmica*, 12(1):18–29, 1994.

[10] Mordecai J. Golin, Rajeev Raman, Christian Schwarz, and Michiel H. M. Smid. Randomized data structures for the dynamic closest-pair problem. *SIAM J. Comput.*, 27(4):1036–1072, 1998.

[11] Junhao Gan and Yufei Tao. DBSCAN revisited: Mis-claim, un-fixability, and approximation. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, Melbourne, Victoria, Australia, May 31 - June 4, 2015*, pages 519–530. ACM, 2015.

[12] Zhitao Shen, Muhammad Aamir Cheema, Xuemin Lin, Wenjie Zhang, and Haixun Wang. Efficiently monitoring top-k pairs over sliding windows. In *IEEE 28th International Conference on Data Engineering (ICDE 2012), Washington, DC, USA (Arlington, Virginia), 1-5 April, 2012*, pages 798–809. IEEE Computer Society, 2012.

[13] Michiel H. M. Smid. Maintaining the minimal distance of a point set in polylogarithmic time. In Alok Aggarwal, editor, *Proceedings of the Second Annual ACM/SIGACT-SIAM Symposium on Discrete Algorithms, 28-30 January 1991, San Francisco, California, USA*, pages 1–6. ACM/SIAM, 1991.

[14] Sanjiv Kapoor and Michiel H. M. Smid. New techniques for exact and approximate dynamic closest-point problems. *SIAM J. Comput.*, 25(4):775–796, 1996.

[15] Yiqiu Wang, Shangdi Yu, Yan Gu, and Julian Shun. A parallel batch-dynamic data structure for the closest pair problem. In *37th International Symposium on Computational Geometry, SoCG 2021, June 7-11, 2021, Buffalo, NY, USA (Virtual Conference)*, volume 189 of *LIPIcs*, pages 60:1–60:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[16] Pankaj K. Agarwal, Haim Kaplan, and Micha Sharir. Kinetic and dynamic data structures for closest pair and all nearest neighbors. *ACM Trans. Algorithms*, 5(1):4:1–4:37, 2008.

[17] David Eppstein. Fast hierarchical clustering and other applications of dynamic closest pairs. *ACM J. Exp. Algorithmics*, 5:1, 2000.

[18] Jean Cardinal and David Eppstein. Lazy algorithms for dynamic closest pair with arbitary distance measures. In *Proceedings of the Sixth Workshop on Algorithm Engineering and Experiments, New Orleans, LA, USA, January 10, 2004*, pages 112–119. SIAM, 2004.

A PROOF OF LEMMA/THEOREM

LEMMA 2. *Given a streaming dataset $\mathcal{S}$, let R_i and R_{i+1} (= $R_i \cup \{p(o, o')\}$) be the two result sets to KCP queries corresponding to $k = i$ and $k = (i+1)$, respectively. It is guaranteed that if $o \notin R_i.obj \vee o' \notin R_i.obj$, $p(o, o') \in \mathcal{N} - R_i$.*

PROOF. If $o \notin R_i.obj$, $p(o, NN(o)) \notin R_i$. Based on the fact that $o \in R_{i+1}.obj$, o' must be $NN(o)$. Follow the same logic, if $o' \notin R_i.obj$, $NN(o')$ must be o. Therefore, $p(o, o')$ belongs to $\mathcal{N} - R_i$. The proof completes. □

LEMMA 3. *Let $e_i.O$ be the set of objects in e_i, $c_{max}(\eta_{\frac{1}{2}})$ be the max cube constructed based on objects in $e_i.O$, $e_i.c$ be the standard cube with minimal side-length that bounds objects in $e_i.O$, and c be any standard cube in the space of $e_i.c$ not covering $c_{max}(\eta_{\frac{1}{2}})$. It is guaranteed that both c and $c_{max}(\eta_{\frac{1}{2}})$ contain* no *more than $\frac{|e_i.O|}{2}$ objects.*

PROOF. Since neither $\eta_{\frac{1}{4}}$ nor $\eta_{\frac{3}{4}}$ is within $c_{max}(\eta_{\frac{1}{2}})$, at least $\frac{|e_i.O|}{2}$ objects are out of $c_{max}(\eta_{\frac{1}{2}})$. Thus, $c_{max}(\eta_{\frac{1}{2}})$ contains no more than $\frac{|e_i.O|}{2}$ objects. For each standard cube c that is within the space $e_i.c$ and meanwhile does not contain $c_{max}(\eta_{\frac{1}{2}})$, it does not contain $\eta_{\frac{1}{2}}$. Accordingly, $\exists i \in [1, d]$ such that i) $c[i] > \eta_{\frac{1}{2}}[i]$ or ii) $c[i] + |c| < \eta_{\frac{1}{2}}[i]$. For both case (i) and case (ii), there are at least $\frac{|e_i.O|}{2}$ objects are outside c. Thus, c also contains no more than $\frac{|e_i.O|}{2}$ objects. □

LEMMA 4. *Given $C_\delta = \{c \in C | |c| \leq \delta\}$, there must exist a standard cube with side length 2δ that bounds all the cubes in C_δ.*

PROOF. All the cubes in C, the super set of C_δ, are formed by mapping rules (1) or (2). Accordingly, we partition cubes in C_δ into two disjoint subsets, C_δ^1 containing the cubes generated by rule (1) and C_δ^2 containing cubes generated by rule (2). For cubes c_i in C_δ^1, c_i is a standard cube of side length $\frac{|c_i'|}{2}$, with standard cube c_i' being the minimal standard cube that bounds both o_i and $c_{max}(\eta_{\frac{1}{2}})$ and o_i is located outside $c_{max}(\eta_{\frac{1}{2}})$. In other words, $|c_i| = \frac{|c_i'|}{2} \geq |c_{max}(\eta_{\frac{1}{2}})|$. We form a complementary set $C_\delta^{1'}$ that captures all the standard cube c_i' corresponding to $c_i \in C_\delta^1$. For cubes c_j in C_δ^2, it is guaranteed $c_j = c_{max}(\eta_{\frac{1}{2}})$.

On the other hand, given any two standard cubes c_i and c_j with $|c_i| \geq |c_j|$ and meanwhile both covering $c_{max}(\eta_{\frac{1}{2}})$, it is guaranteed that c_i fully covers c_j. Consequently, since all the cubes in $C_\delta^{1'}$ cover $c_{max}(\eta_{\frac{1}{2}})$, they (and hence all the cubes in C_δ^1) are fully bounded by the standard cube c_j' in $C_\delta^{1'}$ with the largest side-length (i.e., 2δ). This standard cube c_j' covers all cubes in C_δ^2 too. In other words, the standard cube c_j' with side length 2δ covers all the cubes in C_δ. □

THEOREM 1. *Given a non-leaf node e of QC-Tree, let Child(e) represent all its child nodes. If there is one child node $e_i \in Child(e)$ having more than $\frac{e.n}{2}$ objects, it is guaranteed that all the child nodes of e_i have no more than $\frac{e.n}{2}$ objects, i.e., if $\exists e_i \in Child(e)$, $e_i.n \geq \frac{e.n}{2}$, then $\forall e' \in Child(e_i)$, $e'.n \leq \frac{e.n}{2}$.*

PROOF SKETCH: when e is partitioned into 4 quadrants (e_1 to e_4), e_5 is introduced only if the number of objects located inside a quadrant (say e_4) is larger than $\frac{e.n}{2}$. Consequently, we only need to prove that all the child nodes of e_4 and e_5 have no more than $\frac{e.n}{2}$ objects. A $e_5.c$ is the corresponding to c_ξ and $|c_\xi|$ =min($2s_{me}$, $\frac{|e_4.c|}{2}$), c_ξ may contain more objects than e_4 as at least half of objects bounded by e_4 are contained in c_ξ. As each sub-space of c_ξ with side-length $\frac{|c_\xi|}{2}$ contain no more than $\frac{n'}{2}$ ($n' = e_4.n + e_5.n < e.n$) based on the construction of c_ξ, it must contain less than $\frac{e.n}{2}$ objects.

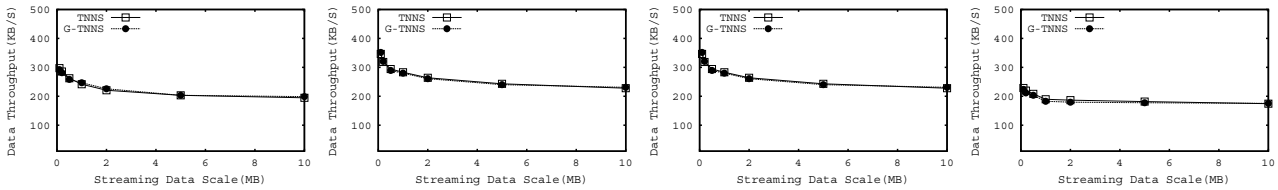
THEOREM 2. *Once all the range queries corresponding to objects in $\mathcal{S}$ are processed, $\forall o \in P_i$, if $o.NN$ is NULL, we have $m_i < \mathsf{D}(o)$; otherwise, $o.NN = NN(o)$.*

PROOF. Given $o \in P_i$, if $o.NN$ is still $NULL$ after processing all the range queries, it is guaranteed that $\mathsf{D}(o) > m_i$ though we have not found $NN(o)$. This can be proved by contradiction. We assume $\mathsf{D}(o) \leq m_i$, and hence there must be an object o' such that $\mathsf{D}(o, o') \leq m_i$. If $o' \in \cup_{j \in [1,i]} P_j$, it will be retrieved by the i range queries issued based on o, as the search ranges of those i queries are no smaller than m_i. Otherwise, $\exists j \in [i+1, m]$ such that $o' \in P_j$. The range query $q_i(o', m_i, P_i)$ issued based on o' on objects in P_i will bound object o and hence update $o.NN$. Given the fact that $o.NN$ is still $NULL$, our assumption $\mathsf{D}(o, o') \leq m_i$ is invalid and hence $\mathsf{D}(o) > m_i$. If $o.NN$ is not $NULL$, it means that we have found a real object with $\mathsf{D}(o, o.NN)) \leq m_i$. It is guaranteed that $o.NN = NN(o)$, which can be easily proved following the logic we use to prove the above case. □

THEOREM 3. *Given an object o to be remapped/inserted to a partition, assume partition P_i is the first partition such that $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$ and o is inserted into $P_j (1 < j < m)$. It is guaranteed $m_j \leq \mathsf{D}(o) \leq \mu m_{j-1}$ with μ bounded by $(1 + \sqrt{d})$.*

PROOF. As stated previously, when we insert object o to a proper partition, there are in total two cases when we find a partition P_i ($i < m$) such that $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$. To facilitate the following proof, let o' and o'' be the object in the first i partitions $\cup_{l=1}^{i} P_l$ and next $(m - i)$ partitions $\cup_{l=i+1}^{m} P_l$ having the shortest distance to o respectively. In other words, it is guarantee that i) $\forall obj \in \cup_{l=1}^{i} P_l$, $\mathsf{D}(obj, o) \geq D(o', o)$ and ii) $\forall obj' \in \cup_{l=i+1}^{m} P_l$, $\mathsf{D}(obj', o) \geq \mathsf{D}(o'', o) \geq m_i$.

In the first case, $\mathsf{D}(o, o') < m_i$ and o is inserted into a partition P_j ($j > i$) such that $m_j \leq \mathsf{D}(o, o') < m_{j-1}$. Under this case, as $D(o'', o) \geq m_i$, o' must be $NN(o)$,i.e., $\mathsf{D}(o)$=$\mathsf{D}(o, o')$. We have $m_j \leq \mathsf{D}(o, o')(=\mathsf{D}(o)) < m_{j-1}$ with $\mu = 1$. In the second case, $\mathsf{D}(o, o') \geq m_i$, $C_i^{q_i} = \emptyset$ or $\sum_{c \in C_i^{q_i}} c.n = 0$. o is inserted into P_i. It's guaranteed that $\mathsf{D}(o) \geq m_i$. As the search for partition P stops at P_i, there is at least one standard cube c of C_{i-1} associated with partition P_{i-1} that overlaps with the search range of q_{i-1} and $c.n > 0$. As $c.n > 0$, there is at least one object o''' in the subsequent partitions $\cup_{l=i}^{m} P_l$ fallen with c. As c overlaps with the search region, the minimal distance between o and c is no larger than m_{i-1}. Accordingly, the maximal distance between o and c is no larger than $m_{i-1}(1 + \sqrt{d})$. Therefore, we have $\mathsf{D}(o, o''') \leq (1 + \sqrt{d}) m_{i-1}$, i.e., $m_i \leq \mathsf{D}(o) \leq (1 + \sqrt{d}) m_{i-1}$ with $\mu = 1 + \sqrt{d}$ and $j = i$. □



(a) Stock ($s = 0.1\% \times n$) (b) Trip ($s = 0.1\% \times n$) (c) Uniform ($s = 0.1\% \times n$) (d) Normal ($s = 0.1\% \times n$)

Figure 8: Data throughput comparison of TNNS under Guard under different data sets under different n

B THE LOCAL RE-PARTITION VIA GUARD

As we mention previously, the split and merge of the last partition P_m, in an extreme case, might happen frequently which incurs a very high maintenance cost of $\tau-$DLBP partition. In order to lower down the update cost and mitigate the risk of frequent change of P_m, we propose a new data structure namely *Guard*, formally defined below. Note, $P_i.\theta$ refers to the corresponding $g(\tau)$ used by TNNS when P_i is the last partition that covers all the objects in TNNS; $P_i.O$ refers to the set of objects in P_i with scores between $(\mathsf{LB}(P_i), P_i.\theta]$ (i.e., those objects in P_i but not in TNNS); $P_i.P_{nn}$ refers to the set of corresponding NN pairs formed by objects in $P_i.O$; and $P_i.t$ refers to the time stamp when P_i is split and changes from the last partition to the second last partition.

DEFINITION 5. **Guard**. *A guard, expressed as a tuple $\langle \theta, O, P_{nn}, t\rangle$, is associated with a partition P_i, iif P_i is used to be the last partition of $\tau-$DLBP at a time stamp within the window $[t_{cur} - |P_i.O|, t_{cur}]$ with t_{cur} referring to the current time stamp.*

Let $\mathcal{S}$ be partitioned into $\{P_1, P_2, \cdots, P_m, P_{m+1}\}$ and P_m be a partition associated with a guard structure. When a few objects in P_{m+1} expire, local re-partition might be triggered because of condition i) or iii). For both cases, we need to find the scores of objects in P_{m+1} and P_m. Instead of relying on range queries to find objects' scores, we could utilize $P_m.P_{nn}$ maintained by the guard structure. In the following, we will discuss guard maintenance under data stream.

Guard forming and deconstructing. A guard is formed at the moment (say t_m) the last partition P_m of $\tau-$DLBP is split into P_m and P_{m+1}. In order to efficiently maintain the guard of the partition P_m, side-length of cubes in the corresponding cube set C_m is set to $P_i.\theta$ other than $\mathsf{LB}(P_i)$. Along with $\mathcal{S}$ updates, the guard associated with P_m is deconstructed at the moment $t_m + |P_m.O|$, assuming the total number of partitions is larger than m from t_m to $t_m + |P_m.O|$. Accordingly, side-length of cubes in C_m is updated to $\mathsf{LB}(P_m)$. The intuition behind it is if the last partition is not frequently split *or* merged, even we do not maintain the *Guard* structure, the amortized local re-partition cost is not high. So, after $|P_m.O|$ updates, if the current P_m is still not used for forming TNNS, we deconstruct the *Guard* structure so as to reduce the space cost.

Guard maintenance under data stream. Given a dataset $\mathcal{S}$ partitioned into $\{P_1, P_2, \ldots, P_r\}$, let P_i be the partition with smallest index i associated with a guard structure. When a new object o arrives in $\mathcal{S}$, we process o based on the algorithm presented in Section 4.3. When we evaluate partition from P_1 to P_{i-1}, if there is an object o' in $P_j (j < i)$ having distance to o between $\mathsf{LB}(P_i)$ and $P_i.\theta$, we update guard of P_i based on o'. When evaluating P_i, we submit a range query $q(o, P_i.\theta, P_i)$, and then update guard of P_i based on $R_i(o)$. If the search is terminated after accessing P_i, $\mathsf{D}(o)$ is also bounded. The proof is similar with that is stated Theorem 3, we skip the details. When an object o_{exp} expires from a partition P_i that is associated with a guard structure, if $o_{exp}.d > P_i.\theta$, we remove it from P_i. Otherwise, we process its reverse nearest neighbours following the logic of processing expired objects in $T.obj$,i.e., updating guard of P_i based on distance among o_{exp}'reverse nearest neighbours and their new nearest neighbours.

Experiment Under GUARD. In the following, we supplement a set of experiments, where we evaluate the performance of TNNS under guard, denoted as GTNNS. As the performance of TNNS without guard performs much better than the others, we only compare GTNNS with TNNS. We report the data throughput of all the algorithms under different n values in Figures 8(a)-8(d). We can observe that the data throughput of GTNNS is roughly as the same as that of TNNS. The reasons are as follows. Firstly, τ-DLBP guarantees that TNNS only maintains a small number of NN pairs, compared with the overall running cost of processing all objects in the streaming data set, the proportion of TNNS maintenance cost is small. Secondly, the frequency of local re-partition is not high. Our propose algorithm guarantees that local re-partition happens only when the distribution of streaming data is drastically changed,i.e., many NN pairs with small scores are formed or deconstructed. Last but not least, the extreme case discussed in Section 4.4 is almost not happened in real applications. Therefore, no matter whether guard is used for maintaining $\tau-$DLBP, we also could efficiently support KCP search.

C COST ANALYSIS

QC-Tree Construction Cost Analysis. Given a QC-Tree $\mathcal{I}$, let e be a non-leaf node. When we form its child nodes, it first requires $O(d \cdot e.n)$ to partition the space (and objects) into 2^d quadrants. If the construction of e_5 is required (i.e., one of the child node e_i has at least $e.n/2$ objects), it incurs additional running cost to form $\eta_{max}(\frac{1}{2})$ and c_{ξ}. The former requires quantile-search whose running cost is linear to the data set scale with hidden constant d; the latter also involves quantile-search with computational cost bounded by $O(e.n)$. Given the fact that there are in total $e.n$ objects in e, the running cost of forming its child nodes per object is $O(d)$. As the height of QC-Tree is bounded by $O(\log n)$, there are at most $O(\log n)$ nodes containing any given object o. Accordingly, the QC-Tree construction running cost is bounded by $O(d \cdot n \cdot \log n)$. Once QC-Tree is formed, L Cube of each object is formed accordingly. Thus, the overall running cost of L Cube construction is $O(d \cdot n \cdot \log n)$.

$\tau-$DLBP Construction Cost Analysis. The $\tau-$DLBP construction involves three steps, including L Cube-based partition, score lower-bound calculation and refinement. For the first step, the running cost of constructing L Cubes is bounded by $O(dn \log n)$. As we use quantile search based algorithm to form the partition, the running cost of L Cube-based partition is bounded by $O(n)$. For the second step, each object $o \in P_i$ submits i range queries. When o searches on P_j $(1 \leq j \leq i)$, the search region overlaps with 3^d cubes. If $i, j \neq m$, each cube contains at most one object. The running cost spent on one range query is bounded by $O(3^d)$, and hence the total running cost spent on objects in P_i is bounded by $O(3^d i|P_i|)$. Otherwise (*i.e.*, $i = j = m$), in an extreme case, as side-length of 2τ objects' L Cubes is no larger than $|c^{2\tau}|$, such objects may be packed in *one* cube with side-length m_m and the range query cost per object is bounded by $O(3^d + \tau)$. In normal cases, at most 3^d objects in P_m are located inside neighbour cubes of c, and hence the running cost per object is also $O(3^d + \tau)$. Other objects in P_m spend $O(3^d)$ running cost. Totally, running cost spent on range queries is bounded by $O(\sum_{i=1}^{i=m} 3^d i|P_i| + 3^d(|P_m| - 3^d) + 3^d(3^d + \tau) + \tau(\tau + 3^d))$. As $|P_i| \leq \frac{n}{2^i}$, the running cost equals to $O(\sum_{i=1}^{i=m} i\frac{n}{2^i} + \tau^2)$, which is $O(3^d n + \tau^2)$. For the running cost of refinement, it is $O(n)$. Similarly, the running cost of forming cube sets is also $O(n)$. Thus, the total $\tau-$DLBP construction cost is $O(3^d n + \tau^2 + dn \log n)$.

Local Re-partition Cost Analysis. We first analyze the running cost of local re-partition under condition (i), where we assume that the algorithm is terminated after evaluating objects in $P_i \cup P_m$. The running cost includes: (i) local-L Cube maintenance running cost; and (ii) range queries cost. For the former one, as local-L Cube of every object is formed at most once after $\mathcal{S}$ updates n times and $|P_m| < |P_i|$, the running cost is bounded by $O(d|P_i \cup P_m| \log |P_i \cup P_m|)$. As $|P_i \cup P_m|$ is bounded by n, this part of running cost is bounded by $O(dn \log n)$,i.e., $O(d \log n)$ per object.

For the later one, after we access objects in $P_i \cup P_m$, each object $o \in P_j (j > i)$ goes through two stages. In stage one, we perform local re-partition using objects in the original partitions from P_m to P_{i+1}. During this stage, each object in $P_j (j > i)$ in total submits $j - i$ range queries, where the search range of each query overlaps up to 3^d standard cubes. Therefore, the cube accessing cost spent on each object is $O((j - i)3^d)$. Besides, as the algorithm is not terminated after accessing P_{i+1}, it means the total number of objects contained in query regions of all queries is bounded by $O(\tau)$. As $|P_j|$ is excepted to be $\frac{|P_j|}{2^{j-i}}$, the overall running cost under stage one is $O(\sum_{j=i+1}^{m} 3^d(j-i-1)|P_j|+\tau)$, which is $O(3^d|P_i|+\tau^2)$. In stage two, we use objects in the current P_m and P_i for local re-partition, with objects in $P_i \cup P_m$ mapped into cubes with side-length $\mathsf{LB}(P_{i-1})$. If the number of local-L Cubes with side-length no larger than $\frac{m_{i-1}}{2\sqrt{d}}$ is at least 2τ, it applies $\tau-$DLBP construction algorithm to form the new partitions with running cost bounded by $O(|P_i|3^d + \tau^2)$. Otherwise (i.e., the number is small than 2τ), $2\tau - 1$ objects may fall within *one* cube c with side-length m_{i-1} in the worst case. The range query spent on each of them is bounded by $O(3^d + \tau)$. In the other cases, at most 3^d objects in P_i are located inside the neighbour cubes of c that o is located, and the running cost spent on each of them is $O(3^d + \tau)$. Other objects (i.e., those located outside the neighbor cubes of c) in P_i spend $O(3^d)$ running cost. Accordingly, the total running cost is bounded by $O(|P_i|3^d + \tau^2)$. As local re-partition under case (i) is triggered by expiration of objects in $P_m \cup P_i$, and we use the structure guard to mitigate the risk of frequent change of P_m, this part of cost is bounded by $O(3^d)$ per update. In brief, regardless of the number of such local L Cubes, the cost spent on each local re-partition under condition i) is bounded by $O(\tau + 3^d + d \log n)$ per object.

In the following, we analyze the cost of local re-partition under conditions (ii) and (iii). Under condition (ii), when P_m is split, as we have computed nearest neighbours for all the objects in P_m, we can use median search algorithm to partition the original P_m into P_m and P_{m+1} with cost bounded by $O(|P_m|)$. Under condition (iii), assuming we merge P_i into P_{i+1}, the running cost is bounded by $O(|P_i| + |P_{i+1}|)$, i.e., $O(1)$ per object.

The incremental maintenance cost. The arrival of new object(s) triggers in total three operations, as detailed below. First, when an object o flows into S, we assume it is finally inserted into $P_i (j < m)$, with the running cost bounded by $O(3^d j)$. However, if o is inserted into P_m, the running cost is $O(3^d \log \frac{n}{\tau} + \tau)$. Second, the arrival of o reduces the score of another o'. Say $m_{i'} \leq D(o') < m_{i'-1}$, and object o' is moved from P_i to $P_{i'}$ with running cost being $O(i' - i)$. Third, local re-partition under condition (ii) or (iii) happens. As discussed before, each object spends $O(1)$ running cost per time. After n newly arrived objects, assuming all objects are inserted into the last partition P_m, the total running cost is bounded by $O(3^d n \log \frac{n}{\tau} + n\tau)$,i.e.,$O(3^d \log \frac{n}{\tau} + \tau)$ per object.

Similarly, the expiry of an object triggers three different operations too. First, when an object $o_{exp} \in P_i$ expires from $\mathcal{S}$, it is removed from P_i. At the same time, for each cube c with side-length $m_j (j < i)$ that bounds o_{exp}, we set $c.n$ to $c.n - 1$. Second, it may trigger local-L Cubes maintenance with cost bounded by $O(d \log n)$. Third, it may trigger local re-partition under condition (i) and condition (iii). As discussed before, after n updates, the local L cube of each object is formed once at most with running cost bounded by $O(d \log n)$. Assuming all deletions are from the last partition, the total local re-partition running cost under case (i) is bounded by $O(3^d \log \frac{n}{\tau} + d \log n + n\tau)$ after n updates. As the running cost of local re-partition under condition (iii) is $O(1)$ per object, the total running cost of maintaining $\tau-$DLBP is bounded by $O(3^d \log \frac{n}{\tau} + \tau + d \log n)$ per object.

We now analyze the space cost of $\tau-$DLBP. Let $C_i (i \in [1, m])$ be the set of standard cubes with side-length m_i that bound objects in $\cup_{l=i}^{m} P_l$. In the worst case, each cube bounds one object in $\cup_{l=i+1}^{m} P_l$. Thus, $|C_i|$ is bounded by $\sum_{l=i}^{m} |P_l|$. As $|P_i| > 2|P_{i+1}|$, we have $|P_i| > \sum_{i+1}^{l=m} |P_l| > \frac{\sum_{i}^{l=m} |P_l|}{2}$. Thus, $|C_i|$ is bounded by $2|P_i|$, and $\sum_{i=1}^{m} |C_i|$ is bounded by $2\sum_{i=1}^{m} |P_i|$, which is $O(n)$. Besides, each cube consumes $O(1)$ space cost. The overall space cost of $\tau-$DLBP is bounded by $O(n)$.